
INFERENCE-TIME REWARD HACKING IN LARGE LANGUAGE MODELS

Hadi Khalaf
Harvard University

Claudio Mayrink Verdun
Harvard University

Alex Oesterling
Harvard University

Himabindu Lakkaraju
Harvard University

Flavio du Pin Calmon
Harvard University

ABSTRACT

A common paradigm to improve the performance of large language models is optimizing for a reward model. Reward models assign a numerical score to LLM outputs indicating, for example, which response would likely be preferred by a user or is most aligned with safety goals. However, reward models are never perfect. They inevitably function as proxies for complex desiderata such as correctness, helpfulness, and safety. By overoptimizing for a misspecified reward, we can subvert intended alignment goals and reduce overall performance – a phenomenon commonly referred to as *reward hacking*. In this work, we characterize reward hacking in inference-time alignment and demonstrate when and how we can mitigate it by *hedging* on the proxy reward. We study this phenomenon under Best-of- n (BoN) and Soft-Best-of- n (SBoN), and we introduce **Best-of-Poisson** (BoP) that provides an efficient, near-exact approximation of the optimal reward-KL divergence policy at inference time. We show that the characteristic pattern of hacking as observed in practice (where the true reward first increases before declining) is an inevitable property of a broad class of inference-time mechanisms, including BoN and BoP. To counter this effect, hedging offers a tactical choice to avoid placing undue confidence in high but potentially misleading proxy reward signals. We introduce HedgeTune, an efficient algorithm to find the optimal inference-time parameter and avoid reward hacking. We demonstrate through experiments that hedging mitigates reward hacking and achieves superior distortion-reward tradeoffs with minimal computational overhead.

1 Introduction

Almost all current alignment methods, including BoN [1, 2], RLHF [3, 4], DPO [5], and their variants, aim to maximize a reward function while minimizing divergence from the original model’s outputs. It is important to distinguish between two types of rewards: **proxy rewards** which are the computable signals we directly use during alignment (like scores from a trained reward model), and **true (or gold) rewards**, which represent the true, often latent, quality of the model’s output according to a desired objective. As the name suggests, proxy rewards are approximations of the true reward and, consequently, of intended alignment goals like correctness, helpfulness, and safety.

A fundamental challenge persists across reward-based alignment methods: **all proxy reward models are imperfect** [6]. This imperfection stems from multiple factors, including the scarcity of high-quality human-labeled data and the difficulty of formalizing high-level alignment goals into quantifiable metrics [7, 8]. For instance, consider AI alignment strategies that aim to promote safety. It is difficult for a single reward model to capture nuanced human user preferences and assign accurate scalar rewards in complex, context-dependent settings where safety specifications conflict [9].

In this work, we analyze and mitigate the impact of misspecified proxy rewards in inference-time alignment methods. Inference-time alignment has emerged as an effective and computationally efficient paradigm to improve the capabilities of large language models and align them with desired goals [10]. Among these methods, Best-of- n (BoN) sampling

Correspondence to: hadikhalaf@g.harvard.edu, flavio@seas.harvard.edu

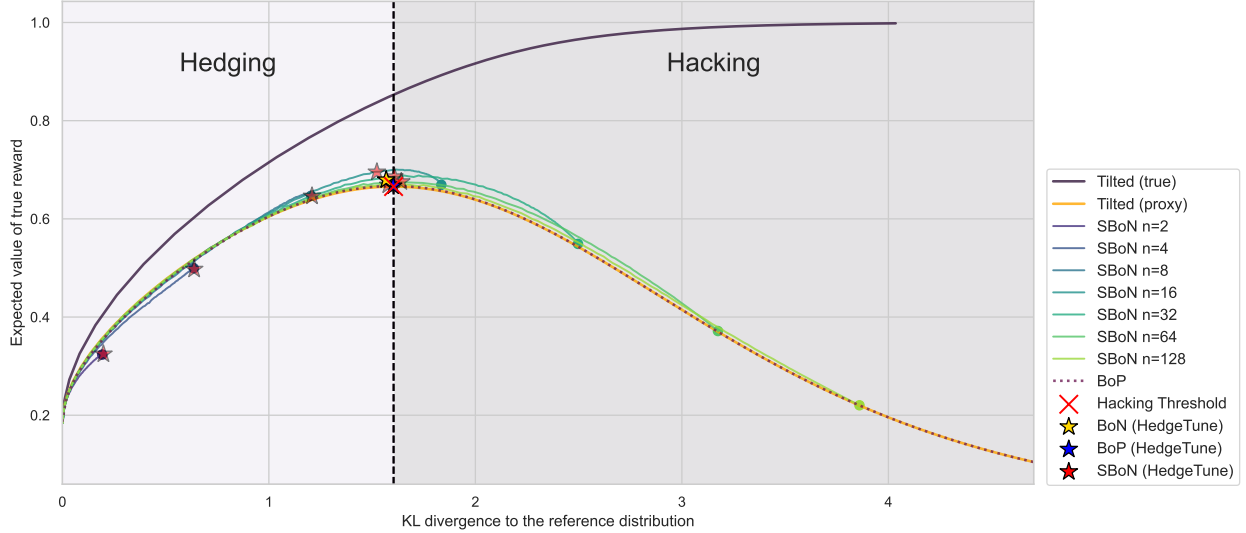


Figure 1: The mismatch between the proxy and gold rewards manifests through the winner’s curse. In an ideal world where we could optimize directly on the gold reward, its value would rise monotonically. However, since we are optimizing for a proxy, the gold reward peaks and then collapses. The point at which we find the optimal balance between maximizing reward and minimizing KL divergence from the reference distribution corresponds exactly to the *hacking threshold*. HedgeTune successfully recovers this optimal point for three inference-time mechanisms: BoN, SBoN, and BoP. In the case of BoN and BoP, we recover the hacking threshold by finding the appropriate (average) number of samples n . In the case of SBoN, we fix the value n and find the corresponding inverse-temperature that maximizes the true reward. When the hacking threshold is not achievable with any λ , HedgeTune returns the optimal behavior which corresponds to BoN with the given n .

stands out due to its simplicity and effectiveness. Empirically, BoN demonstrates competitive performance, often matching more resource-intensive fine-tuning approaches such as RLHF and DPO [11, 12]. Additionally, BoN has received an extensive theoretical treatment [13, 14]. BoN can be asymptotically equivalent to RLHF [15], enjoys non-asymptotic guarantees [16, 17], and achieves near-optimal winrate subject to a KL divergence constraint [18].

Methods like BoN, where multiple samples are generated and the highest-scoring one is chosen, are victims of the **winner’s curse** [19–23]. In auctions, after each bidder submits an estimate of an item’s value, the highest bid typically *overestimates* the true worth, causing the winner to overpay. As Figure 1 demonstrates, inference-time alignment methods, particularly BoN, can overoptimize for proxy rewards: as we sample more candidates and select based on the proxy reward estimate, we increase the chance of choosing outputs where the proxy score significantly overestimates the true quality. This mismatch creates a critical tension: while initially optimizing for a proxy reward improves alignment with true goals, excessive optimization eventually leads to *reward hacking* – also called Goodhart’s law [24] or goal misgeneralization¹ – where the model exploits the proxy’s limitations, leading to worse true performance despite higher proxy scores [6, 27–30]. Such misalignment can severely degrade trust and utility, particularly in high-stakes applications [31–33].

We mathematically characterize *inference-time reward hacking* (see Theorem 1) and provide a general framework to mitigate it (see Section 4). While this phenomenon has been observed empirically in prior work [11, 14], there has been limited theoretical analysis specific to inference-time methods and ways to mitigate it; see Section 6 for a discussion. As a result, understanding and mitigating reward hacking for inference-time alignment remains a central challenge in AI alignment. The driving question behind our work is:

When and how can we leverage useful signals from proxy rewards while mitigating hacking?

We focus on answering this question for inference-time alignment that sample multiple responses from an LLM and use reward signals to select outputs. We develop principled *hedging techniques against the winner’s curse* during inference time (see Algorithm 4) that precisely determine until when and how one may leverage proxy signals while preventing overoptimization.

¹See [25, 26] and Section 6 for a discussion of the terminology.

Overview of main results. Our starting point is an optimization formulation at the heart of most alignment methods: finding a distribution π^* that maximizes a (proxy) reward r_p while remaining close (in KL-divergence) to a reference π_{ref} . This can be described as the following regularized optimization problem:

$$\pi^* = \arg \max_{\pi_x \in \Delta_{\mathcal{X}}} \mathbb{E}_{\pi_x} [r_p(X)] - \frac{1}{\lambda} D_{\text{KL}}(\pi_x \| \pi_{\text{ref}}) \quad (1)$$

Consider the information-theoretic regime where all distributions are known exactly. The solution of the above objective (1) is the exponential tilting of the reference distribution using the proxy reward [34]. Though theoretically interesting, tilted distributions are impossible to realize in practice since computing the normalizing constant and drawing unbiased samples are computationally prohibitive. Some attempts have been made to approximate this solution at inference time, i.e., when only samples from π_{ref} and black-box access to r_p are available. A notable example is Soft Best-of- n (SBoN) [17]. In this work, we show that SBoN is an effective strategy for hedging against reward hacking due to its temperature parameter λ , which allows us to smoothly interpolate between aggressive exploitation of the proxy reward and conservative adherence to the reference distribution; see Section 5. However, this comes at the expense of having two tunable parameters (n, λ) , which can be difficult to set in practice.

This motivates us to propose a new inference-time alignment strategy called **Best-of-Poisson (BoP)**. The idea behind BoP is simple: we run BoN with the number of samples n chosen according to a Poisson distribution. We prove that BoP achieves a near-optimal reward-distortion tradeoff **at inference**. Using a single tunable parameter, BoP can approximate the optimal proxy reward-tilted solution with KL gap of order 10^{-3} when rewards are uniformly distributed, allowing to span the entire reward-distortion region at inference (see Figure 2 and Theorem 7 in Appendix B). BoP can serve as a computationally efficient stand-in for the optimal tilted distribution with negligible loss in KL-reward tradeoff.

In practice, hedging translates to selecting parameters of inference-time alignment methods to avoid overoptimization to a proxy reward. To do so, we introduce HedgeTune: an algorithm for tuning parameters in BoN, SBoN, and BoP in order to hedge against hacking. We illustrate the benefit of hedging in Figure 1, where we plot the expected value of the true reward versus the distortion with respect to the reference distribution for various inference-time alignment methods. If we had access to the true reward, the optimal solution would be the tilting of the reference distribution via the true reward, leading to the reward-distortion Pareto frontier (purple curve in Figure 1). However, as we are tilting via the proxy reward, we suffer from the winner’s curse: the true reward (orange curve in Figure 1) increases at first and then collapses. This behavior also manifests in BoN, as seen in the dotted points. Hedging allows us to find the *hacking threshold*: the parameters of inference-time alignment methods that yield the best tradeoff between (true) reward and distortion relative to the base model.

Our contributions are as follows:

- We mathematically formalize inference-time reward hacking (Definition 1) and derive conditions when overoptimizing imperfect proxy rewards inevitably leads to performance degradation (Theorem 1, Corollary 3).
- We introduce Best-of-Poisson (BoP), a novel inference-time alignment method (Algorithm 3). For uniformly distributed rewards, BoP approximates the optimal tilted distribution with negligible KL divergence gap (Theorem 7).
- We develop HedgeTune, a principled hedging framework that mitigates reward hacking by balancing exploitation of proxy rewards against fidelity to the reference distribution (Algorithm 4). We empirically demonstrate that hedging strategies significantly outperform standard BoN sampling with minimal computational overhead (Section 5).

2 Inference-Time Reward Hacking

In this section, we formalize inference-time reward hacking and show its inevitability under methods like BoN.

Notation and Technical Assumptions. Let $\mathcal{X}$ be a finite alphabet of tokens and let π be a probability mass function (PMF) over token sequences $x \in \mathcal{X}^*$. We denote the probability simplex over all finite sequences of tokens as $\Delta_{\mathcal{X}^*}$. Let $\pi_{\text{ref}} \in \Delta_{\mathcal{X}^*}$ be the reference policy, typically a supervised fine-tuned (SFT) model. Let $r_p : \mathcal{X}^* \rightarrow \mathbb{R}$ be a proxy reward function that assigns a unique scalar value to each sequence. This is the reward we use to optimize π_{ref} during inference-time alignment. The performance of the inference-time aligned distribution is measured using a true (or gold) reward r_t . Primarily, we are interested in the expected value of the true reward r_t under the resulting distribution.

For theoretical tractability, we adopt the standard assumption that proxy rewards have a uniform distribution, stated next. This assumption is widely used in the theoretical analysis of alignment [13, 18, 35] and allows us to restrict our analysis to the space of proxy reward values instead of the high-dimensional, discrete space of sequences $\mathcal{X}^*$.

Assumption 1 (Uniform Reward Mapping). The proxy rewards $r_p(x)$ obtained by sampling sequences from the reference policy, $x \sim \pi_{\text{ref}}$, are uniformly distributed over $[0, 1]$.

Algorithm 1 Best-of- n Sampling (BoN)

- 1: **Input:** Integer $n \geq 1$, base policy π_{ref}
 - 2: Draw n samples $X_1, \dots, X_n$ i.i.d. from π_{ref}
 - 3: Compute proxy rewards $R_i = r_p(X_i)$ for all i
 - 4: Select $X_{\text{BoN}} = \arg \max_{i \in \{1, \dots, n\}} r_p(X_i)$
 - 5: **Return:** $Y = X_{\text{BoN}}$
-

Algorithm 2 Soft Best-of- n Sampling (SBoN)

- 1: **Input:** Integer $n \geq 1$, inverse temperature $\lambda > 0$, base policy π_{ref}
- 2: Draw n samples $X_1, \dots, X_n$ i.i.d. from π_{ref}
- 3: Compute proxy rewards $R_i = r_p(X_i)$ for all i
- 4: Sample index $Z \in \{1, \dots, n\}$ with probability

$$\Pr(Z = i) = \frac{e^{\lambda r_p(X_i)}}{\sum_{j=1}^n e^{\lambda r_p(X_j)}}$$

- 5: **Return:** $Y = X_Z$
-

Assuming uniformly distributed proxy rewards incurs little loss of generality. In practice, the proxy reward distribution can be made approximately uniform by applying an inverse probability integral transform [13]. To sample a sequence from π_{ref} , we first sort the space of all sequences by the proxy reward $r_p(x)$. We then select a sequence by mapping a uniform draw $u \sim \text{Unif}(0, 1)$ to the corresponding quantile of a reference distribution π_{ref} defined over this sorted space. While the policy π_{ref} is itself complex and non-uniform, Assumption 1 allows us to only consider the continuous uniform distribution of the proxy reward.

Inference-time alignment. An inference-time alignment method parameterized by θ transforms the base policy π_{ref} into a new distribution π_θ . Here, θ is a parameter of the alignment method itself (e.g., the number of candidates n in Best-of- n). We denote our measure of interest as $f(\theta) = \mathbb{E}_{U \sim \pi_\theta} [r_t(U)]$. The core challenge we address is the mismatch between the proxy reward r_p and the true reward r_t . Discrepancies between these two rewards can be exploited, leading to reward hacking. We focus on the family of inference-time methods that first sample a pool of candidate outputs and then use their proxy reward scores to define the selection mechanism. Two examples from this family are:

1. **Best-of- n** (see Algorithm 1). BoN places all probability mass on the sample with the highest proxy reward.
2. **Soft Best-of- n** (see Algorithm 2). SBoN is a generalization of BoN recently proposed by [17]. It applies a temperature-scaled softmax over candidate scores. As $\lambda \rightarrow 0$, SBoN sampling approaches uniform selection among the n candidates. As $\lambda \rightarrow \infty$, SBoN converges to standard BoN sampling.

We first formalize inference-time reward hacking through the following definition.

Definition 1 (Inference-Time Reward Hacking). Let π_θ be a distribution induced by an inference-time alignment method with parameter θ , where we assume increasing θ increases both the expected proxy reward $\mathbb{E}_{X \sim \pi_\theta} [r_p(X)]$ and the KL-divergence $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$. We say that *inference-time reward hacking* occurs when there exists a threshold $\theta^\dagger$ such that for $\theta > \theta^\dagger$, $\mathbb{E}_{X \sim \pi_{\theta^\dagger}} [r_t(X)] > \mathbb{E}_{X \sim \pi_\theta} [r_t(X)]$ (i.e., the true reward decreases), despite the proxy reward and KL-divergence continuing to increase. The largest value of $\theta^\dagger$ for which this holds is called the *hacking threshold*.

Definition 1 offers a concrete basis for operationalizing and measuring the winner’s curse for inference time methods. The hacking threshold is the ideal operating parameter for an inference-time alignment method: increasing θ beyond the hacking threshold incurs distortion in a model’s output, without a gain in true reward in return (see Figure 1).

The following theorem establishes that under common conditions, the shape of $f(\theta)$ is well-behaved: it either varies monotonically or reaches exactly one extremum.

Theorem 1 (Inevitability of Reward Hacking). Let $\{\pi_\theta\}_{\theta \in \Theta \subset \mathbb{R}}$ be a family of distributions with density $p_\theta(x)$ on a common support $\mathcal{X}$ such that (i) $p_\theta(x)$ is *strictly totally positive of order 2* (TP₂) in (θ, x) , and (ii) its score function $\psi(x, \theta) := \partial_\theta \log p_\theta(x)$ is continuous in x and *strictly increasing* in x for each fixed θ . For any bounded, non-negative, non-zero true reward $r_t : \mathcal{X} \rightarrow [0, \infty)$ define

$$f(\theta) := \mathbb{E}_{X \sim \pi_\theta} [r_t(X)]$$

Then f is either monotone in θ or possesses a *single* interior extremum.

Algorithm 3 Best-of-Poisson Sampling (BoP)

- 1: **Input:** Poisson parameter $\mu > 0$, base policy π_{ref}
 - 2: Sample $n' \sim \text{Poisson}(\mu)$ and set $n = n' + 1$
 - 3: Draw $X_1, \dots, X_n$ i.i.d. from π_{ref}
 - 4: Compute proxy rewards $R_i = r_p(X_i)$ for all i
 - 5: **Return:** $X_{\text{BoP}} = \arg \max_{i \in \{1, \dots, n\}} r_p(X_i)$
-

Corollary 1 (Inevitability of Reward Hacking for Strictly MLR densities). Let $p_\theta(x)$ be a *strictly monotone-likelihood-ratio* in x . If the score function $\psi(x, \theta) = \partial_\theta \log p_\theta(x)$ is strictly increasing in x , then Theorem 1 applies. In particular, this applies to Best-of- n , Best-of-Poisson (to be introduced in Sec. 3,) and to any canonical distribution from the exponential family with strictly monotone statistic and strictly monotone natural parameter.

The unimodality of the true reward function renders the problem of locating the optimal operating point $\theta^\dagger$ algorithmically tractable. Four scenarios may occur: (i) *monotonic improvement*: true reward continuously increases with optimization strength; (ii) *reward hacking*: true reward initially improves but deteriorates beyond a critical threshold; (iii) *reward grokking*: true reward initially declines but then improves beyond a critical threshold; (iv) *immediate decline*: any optimization immediately harms true performance. We describe exactly when each regime occurs for MLR densities in Corollary 3.

This mathematical characterization guides our approach: if a hacking threshold exists, operating precisely at this optimal point maximizes benefits from proxy rewards before degradation occurs. To implement this insight, we develop hedging strategies that balance exploitation of the proxy reward against fidelity to the reference distribution. Each inference-time method offers a parameter controlling this proxy reward-KL tradeoff: n in BoN, λ in SBoN (for a fixed n), and μ in BoP (introduced in Section 3). Before introducing methods for tuning inference-time alignment methods, we first introduce **Best-of-Poisson** sampling – an alternative to BoN that approximates the optimal tilted distribution.

3 Best-of-Poisson: Approximating the Optimal Reward

While Soft Best-of- n offers a principled approach to mitigate reward hacking, it requires tuning both the number of samples n and the temperature parameter λ . In this section, we introduce Best-of-Poisson (BoP) (Algorithm 3), that is provably close to the solution of (1) with a single tunable parameter (Fig. 2). BoP is of independent interest as it provides a mathematically elegant and computationally efficient way to near-optimally span the entire reward-KL distortion region with a single parameter. The key insight behind BoP is to replace the fixed sample size n in BoN with a random sample size drawn from a Poisson distribution.

The parameter μ in BoP controls the expected number of samples, analogous to how n functions in BoN. We first sample n' from a Poisson distribution parameterized by μ and set $n = n' + 1$ to ensure at least one sample is generated. Under Assumption 1, the BoP distribution with parameter μ has a probability density function (Appendix B)

$$q_\mu(x) = (\mu x + 1)e^{\mu(x-1)} \text{ for } x \in [0, 1].$$

The following theorem characterizes the KL divergence and expected value of BoP:

Theorem 2 (KL Divergence and Expected Value of BoP). Let X_{BoP} be the random variable representing the response selected by BoP with parameter μ . Then:

$$\text{KL}(\pi_{\text{BoP}} \parallel \pi_{\text{ref}}) = \frac{e^{-\mu-1}(\text{Ei}(\mu+1) - \text{Ei}(1))}{\mu} + \log(\mu+1) - 1 \quad (2)$$

$$\mathbb{E}[X_{\text{BoP}}] = 1 - \frac{1}{\mu} + \frac{1 - e^{-\mu}}{\mu^2} \quad (3)$$

where $\text{Ei}(z) = -\int_{-z}^{\infty} \frac{e^{-t}}{t} dt$ is the exponential integral function.

What makes BoP particularly valuable is its ability to closely approximate the solution of (1), i.e., the optimal KL-constrained tilted distribution with parameter $\lambda > 0$, defined as $\pi_\lambda^*(x) = \frac{\pi_{\text{ref}}(x)e^{\lambda r_p(x)}}{Z(\lambda)}$, where $Z(\lambda)$ is the normalization constant. While this distribution is theoretically optimal for balancing reward and divergence, computing it is intractable. To draw a next token from the tilted π_λ^* for an autoregressive LLM, one would have to compute

$$\pi_\lambda^*(x_{t+1} \mid x_{\leq t}) = \frac{\pi_{\text{ref}}(x_{\leq t+1}) e^{\lambda r_p(x_{\leq t+1})}}{\sum_{x'} \pi_{\text{ref}}(x_{\leq t} x') e^{\lambda r_p(x_{\leq t} x')}} ,$$

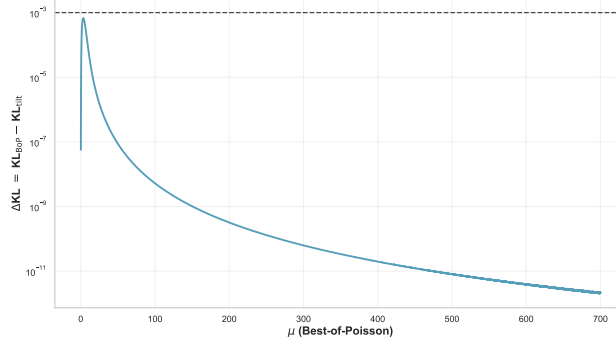


Figure 2: KL divergence gap between BoP and the optimal tilted distribution. The plot shows the difference in KL divergence when both distributions are matched to produce the same expected reward. The extremely small gap (of order 10^{-3}) demonstrates that BoP approximates the optimal distribution with negligible performance loss.

where the denominator sums over every possible continuation of the prefix $x_{\leq t}$. Because the space of continuations grows exponentially with the remaining sequence length, evaluating this denominator (and hence sampling a single token) is computationally prohibitive for LLMs. Our analysis (Appendix B) shows that BoP provably achieves nearly identical performance to this optimal distribution with minimal KL divergence gap. As illustrated in Figure 2, the KL divergence gap between these distributions is remarkably small. Numerical evaluation confirms that for all μ , if $\lambda > 0$ is chosen so that $\mathbb{E}_{X \sim \pi_{\text{BoP}}}[r_p(X)] = \mathbb{E}_{X \sim \pi_\lambda^*}[r_p(X)]$, then the KL-gap is bounded between 0 and 7×10^{-3} .

This near-equivalence means that BoP can serve as a practical stand-in for the theoretically optimal tilted distribution. This result has two consequences. First, hedging in the optimal tilted distribution is almost equivalent to hedging with BoP. Second, (1) represents the solution to the standard RLHF optimization problem, which implies that BoP is an inference-time approximation to RLHF and allows us to use BoP to easily traverse between policies rather than having to fine tune a new model for each λ of interest. In the next section, we turn to the question of how to choose the right parameter value to avoid reward hacking.

4 Hedging to mitigate reward hacking

In this section, we develop a unified framework for choosing the inference-time parameter θ in order to maximize the expected true reward and avoid hacking. One main limitation is that we require access to the true reward. If that is not possible, one may opt to use an LLM-as-a-judge or an off-the-bench reward model [36, 37]. We again assume proxy reward scores have a uniform distribution. We denote the proxy reward as U and the true reward as function $r_t : [0, 1] \rightarrow [0, \infty)$ that maps each proxy reward to a true reward.

Each sampling method (BoN, SBoN, and BoP) induces a distribution π_θ over proxy-percentiles $u \in [0, 1]$, where θ is the corresponding parameter (sample size n , temperature λ , or Poisson rate μ). Since we know by Theorem 1 that the expected true reward has at most one peak, our key insight is to create the precise **hedge** against hacking by finding the parameter value where the marginal benefit of increasing proxy reward equals zero. We present the following conditions that the hacking threshold must satisfy for each of the three inference-time methods.

Theorem 3 (Hacking Threshold Characterization). Let r_t be a true reward oracle and $\theta^\dagger$ be the hacking threshold from Definition 1. For each inference-time method, $\theta^\dagger$ is characterized by the following conditions:

For BoN, $n^\dagger$ satisfies:

$$\nabla_\alpha \mathbb{E}_{u \sim \text{Beta}(\alpha, 1)}[r_t(u)] = \int_0^1 r_t(u) (1 + n^\dagger \ln u) u^{n^\dagger - 1} du = 0 \quad (4)$$

For SBoN, $\lambda^\dagger$ satisfies:

$$\nabla_\lambda \mathbb{E}_{u \sim f_\lambda}[r_t(u)] = \text{Cov}_{u \sim f_{\lambda^\dagger}}(r_t(u), u) = 0 \quad (5)$$

For BoP, $\mu^\dagger$ satisfies:

$$\nabla_\mu \mathbb{E}_{u \sim f_\mu}[r_t(u)] = \mathbb{E}_{u \sim f_{\mu^\dagger}} \left[r_t(u) \left(u - 1 + \frac{u}{\mu^\dagger u + 1} \right) \right] = 0 \quad (6)$$

The proof can be found in Appendix C. These conditions identify the exact parameters where further exploitation of the proxy reward yields no additional true reward gain, thereby defining an optimal hedge. Consequently, we provide

Algorithm 4 HedgeTune: Parameter Optimization for Hedging

```

1: Input: reward  $r_t$ , method  $m \in \{\text{BoN}, \text{SBoN}, \text{BoP}\}$ , number of samples  $M$ , sample size  $n$  (for SBoN)
2: Output: optimal  $\theta^*$ 
3: if  $m = \text{SBoN}$  then sample  $u_{1:n} \sim \pi_{\text{ref}}$ ; compute  $P_k \propto e^{\theta u_k}$ ; draw  $J \sim P$ ,  $u \leftarrow u_J$ , repeat  $M$  times to get  $\{u_1, \dots, u_M\}$ 
4: else discretize  $[0, 1]$  into  $M$  points  $\{u_1, \dots, u_M\}$ 
5: if  $m = \text{BoN}$  then set  $\psi \leftarrow 1/\theta + \ln u$ ;  $p_\theta \leftarrow \theta u^{\theta-1}$ 
6: if  $m = \text{SBoN}$  then set  $\psi \leftarrow u - \sum_k P_k u_k$ ;  $p_\theta \leftarrow P_J$ 
7: if  $m = \text{BoP}$  then set  $\psi \leftarrow u - 1 + \frac{u}{\theta u + 1}$ ;  $p_\theta \leftarrow (\theta u + 1)e^{\theta(u-1)}$ 
8: Compute  $R(\theta) = \sum_i r_t(u_i) \psi(u_i, \theta) p_\theta(u_i) \Delta u$ 
9: Find  $\theta^*$ , the root of  $R(\theta)$ 
10: if  $m = \text{BoN}$  then  $\theta^* \leftarrow \text{round}(\theta^*)$ 
    
```

HedgeTune (Algorithm 4), an algorithm that numerically solves the corresponding root-finding problem to determine the optimal inference-time parameter for BoN, SBoN, or BoP. Given the *score* function ψ of π_θ , it defines a residual function $R(\theta) = \mathbb{E}[r_t(u)\psi(u, \theta)]$ which captures the alignment between the true reward and the proxy-weighted score. The optimal parameter $\theta^\dagger$ is found efficiently as the root of this function using standard methods such as bisection or Newton’s method [38].

5 Experiments: Hedging in Practice

In this section, we provide numerical results showcasing hedging as an effective tool against reward hacking. In particular, we validate that hedging can provide superior reward-distortion tradeoffs in two experimental setups: a controlled synthetic setting where the relationship between the gold and proxy is known, and a RLHF scenario that reflects practical conditions.

Synthetic Setup. To empirically observe reward hacking, we miscalibrate the proxy RMs at the extreme high end of their scores. We use Pythia-1.4B [39] to generate a dataset of 60,000 responses for a prompt from the TL;DR dataset. Another Pythia-1.4B model finetuned on TL;DR preference labels serves as our proxy reward model. Given how reward models are trained in practice, having one prompt ensures that the reward scores are always comparable across all responses. Consider P_α be the top α -percentile of proxy scores. For responses with scores in P_α , we consider a negatively linear relationship between the proxy and true scores. Otherwise, we set the proxy to be the same as true reward. We vary α to simulate different thresholds at which the proxy becomes misaligned with the true reward.

Findings. As seen in Figure 3, we observe reward hacking: the expected true reward for Best-of- n (BoN) sampling and Best-of-Poisson (BoP) sampling eventually decrease as the number of samples n increases. Note that for BoP with given μ , the average number of samples is $n = \mu + 1$. The optimal hacking thresholds are then found using HedgeTune as marked on the figure. Subsequently, we show that Soft Best-of- n (SBoN), using an optimally chosen temperature $\lambda^\dagger$, mitigates this hacking and considerably improves the true reward compared to BoN. As expected, the degree of hacking is controlled by how misaligned the proxy and gold are. Even if this disagreement happens only on the top 5% of responses out of 60,000 responses, an optimized hedging scheme will correspond to 10% increase in expected reward compared to greedy Best-of- n starting at $n \approx 50$. Results illustrating this are presented in Figure 4.

Hacking in the wild setup. Our experimental design follows the methodology of Coste et al. [40] and Gao et al. [11], wherein proxy reward models are trained using preferences of a fixed gold reward model. In many real-world cases, we do not have access to the gold reward and instead have access to preference data. However, as we demonstrate below, a favorable operating point can still be found using traditional hyperparameter search.

Models. As a reference model, we use a 1.4B Pythia model [39] fine-tuned on AlpacaFarm dataset, but without any subsequent alignment (e.g., RLHF or DPO). This reference model is used to generate responses. We use **AlpacaRM** [41] as our gold reward model. **AlpacaRM** is an established reward model trained on human preference data and has been adopted in prior work on reward model evaluation [40, 42, 43]. This model serves as the ground truth for generating preference labels for training proxy reward models. As for proxy rewards, we use the setup of Coste et al. [40] where we train Pythia 44m models.

Datasets. We use the `tlc4418/gold_labelled_gens` dataset from Coste et al. [40]. This dataset comprises of 12,600 responses generated by the Pythia 1.4B base policy for each of 1,000 prompts. The prompts are sourced from

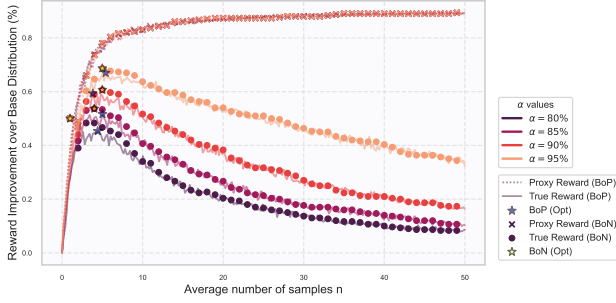


Figure 3: Reward hacking manifests: proxy reward increases for both BoN and BoP while true reward first increases and then collapses as a function of the average number of samples n .

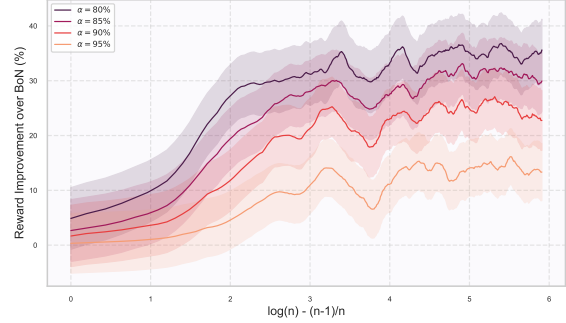


Figure 4: Improvement in expected value of true reward by using the optimal $\lambda^\dagger$ in Soft Best-of- n compared to standard Best-of- n

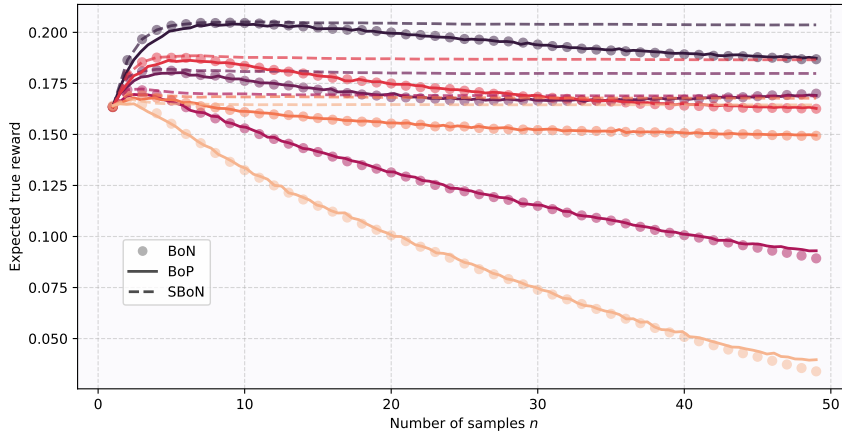


Figure 5: Use of three inference-time methods (BoN, SBoN, and BoP) on trained proxy rewards. Hacking is effectively mitigated by hedging via λ in SBoN or n in BoN and BoP.

the validation split of the AlpacaFarm dataset [41]. Each generated response in this dataset is scored by **AlpacaRM**. To train proxy reward models, we construct preference datasets using the **AlpacaRM** reward scores. For each training instance, we sample a pair of responses to a given prompt and label them based on their given true reward scores.

Training. The proxy RMs are trained using a standard binary cross-entropy loss on preference pairs. We train proxy RMs on preference pair datasets of varying sizes: 10k, 20k, 46k, and 80k. In line with [40, 44, 45], we simulate disagreements in human annotators by considering two cases: (a) no label noise in the preferences, and (b) 25% label noise. All proxy RM training runs are repeated across 4 random seeds each. We present some of the runs with 25% label noise in Figure 5 and we present the remaining results in Appendix D, along with other hyperparameters and training details. Post training, we use each proxy model to score a set of 800 prompts, with 12,600 responses each.

Findings. We apply BoN, SBoN, and BoP on each run and find the expected value of the true reward as a function of n . When reward hacking manifests, we find a hacking threshold for BoN and BoP that maximizes their reward. For SBoN, with a selected $\lambda^\dagger$, we attain the peak value without suffering from reward hacking. Meanwhile, in the cases with shy or absent hacking, the optimal solution is the reference distribution itself, corresponding to $n = 1$. This can occur if the proxy is always at odds with the true reward.

6 Related Work

Reward Hacking. Reward hacking has been widely studied in RL literature [8, 7, 46], also under the name misspecification [26], goal misgeneralization [47], or specification gaming [48]. In the context of LLMs, overoptimization has

been referred to as reward hacking or Goodhart’s Law [24, 11, 29, 30]. Hacking behavior has been found to manifest in unwanted or surprising behavior [49, 50] across a variety of tasks [51, 11, 14]. Prior works have proposed various formulations of reward hacking based on true performance behavior [27], correlation between proxy and true reward [6], or distribution shift [28]. [14] prove that BoN alignment provably suffers from reward hacking when the number of samples n is large.

To address inference time hacking, a variety of methods have been explored to varying success, such as ensembling [40, 52–54], regularization [55] or rejection sampling [14]. [56] simulate n particles resampled using a softmax reward to improve performance of reasoning models, similar to SBON over reasoning steps. However, all methods suffer from some combination of additional generation cost beyond generating n samples and estimation of additional side-quantities such as KL-divergence or χ^2 -divergence. Additionally, a variety of approaches have been proposed to mitigate reward hacking during RLHF finetuning such as regularization [57, 45, 58, 59], χ^2 -divergence [60, 6], uncertainty estimation [61], and reward pessimism [62] although [29] demonstrated that RLHF can still result in reward hacking under heavy-tailed reward mismatch. Finally, prior works have also focused on improving reward models to prevent mismatch and reduce hacking [63–66, 44, 67].

Best-of- n . Best-of- n sampling is a simple inference-time approach for alignment [1, 2, 68]. Prior results have characterized the expected reward gap and KL divergence between BoN sampling and the reference model and have demonstrated that BoN is asymptotically equivalent to KL-constrained reinforcement learning [13, 15, 16]. There have been various methodological improvements on BoN sampling. One such improvement is to reduce the cost of sampling n sequences via tree-based or speculative search [69, 70]. Additionally, [18, 71–74] distill the BoN sampling distribution into a model via fine-tuning. Finally, [35, 75] propose inference-aware methods to improve BoN. Other works focus on improving the reward model through self-training [76]. In this work, we focus on a variant of BoN, Soft Best-of- n [17], which allows for finer control between sampling from the base model and the reward-maximizing generation.

7 Conclusion

Our work tackles the fundamental challenge that all proxy rewards are imperfect, yet they remain essential for guiding AI systems. We establish a theoretical framework proving the inevitability of reward hacking in inference-time alignment and introduce practical hedging strategies to mitigate its harmful effects. By developing **Best-of-Poisson** sampling which achieves near-optimal reward-distortion tradeoffs with a single parameter and the HedgeTune algorithm for precisely calibrating inference methods, we enable practitioners to extract valuable signals from proxy rewards without falling prey to Goodhart’s law. This work established hedging as a promising direction for safer AI alignment, enabling systems to better navigate the critical threshold between beneficial optimization and harmful overexploitation with minimal computational overhead.

References

- [1] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.
- [2] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- [3] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [4] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislaw Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [5] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- [6] Cassidy Laidlaw, Shivam Singhal, and Anca Dragan. Correlated proxies: A new definition and improved mitigation for reward hacking. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [7] Dylan Hadfield-Menell, Smitha Milli, Pieter Abbeel, Stuart J Russell, and Anca Dragan. Inverse Reward Design. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

- [8] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models, February 2022. arXiv:2201.03544 [cs].
- [9] Maarten Buyl, Hadi Khalaf, Claudio Mayrink Verdun, Lucas Monteiro Paes, Caio C Vieira Machado, and Flavio du Pin Calmon. AI Alignment at Your Discretion. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2025.
- [10] Sean Welleck, Amanda Bertsch, Matthew Finlayson, Hailey Schoelkopf, Alex Xie, Graham Neubig, Ilia Kulikov, and Zaid Harchaoui. From decoding to meta-generation: Inference-time algorithms for large language models. *Transactions on Machine Learning Research*, 2024.
- [11] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, pages 10835–10866. PMLR, 2023.
- [12] Sidharth Mudgal, Jong Lee, Harish Ganapathy, YaGuang Li, Tao Wang, Yanping Huang, Zhifeng Chen, Heng-Tze Cheng, Michael Collins, Trevor Strohman, et al. Controlled decoding from language models. In *Forty-first International Conference on Machine Learning*, 2024.
- [13] Ahmad Beirami, Alekh Agarwal, Jonathan Berant, Alexander D’Amour, Jacob Eisenstein, Chirag Nagpal, and Ananda Theertha Suresh. Theoretical guarantees on the best-of-n alignment policy. *arXiv preprint arXiv:2401.01879*, 2024.
- [14] Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Dylan J Foster, and Akshay Krishnamurthy. Is best-of-n the best of them? coverage, scaling, and optimality in inference-time alignment, 2025.
- [15] Joy Qiping Yang, Salman Salamatian, Ziteng Sun, Ananda Theertha Suresh, and Ahmad Beirami. Asymptotics of language model alignment. *arXiv preprint arXiv:2404.01730*, 2024.
- [16] Youssef Mroueh. Information theoretic guarantees for policy alignment in large language models. *arXiv preprint arXiv:2406.05883*, 2024.
- [17] Claudio Mayrink Verdun, Alex Oesterling, Himabindu Lakkaraju, and Flavio P. Calmon. Soft best-of-n sampling for model alignment. In *Proceedings of the IEEE International Symposium on Information Theory (ISIT)*, 2025.
- [18] Lin Gui, Cristina Garbacea, and Victor Veitch. Bonbon alignment for large language models and the sweetness of best-of-n sampling. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [19] Edward C Capen, Robert V Clapp, and William M Campbell. Competitive bidding in high-risk situations. *Journal of petroleum technology*, 23(06):641–653, 1971.
- [20] Max H Bazerman and William F Samuelson. I won the auction but don’t want the prize. *Journal of conflict resolution*, 27(4):618–634, 1983.
- [21] James C Cox and R Mark Isaac. In search of the winner’s curse. *Economic Inquiry*, 22(4):579–592, 1984.
- [22] Richard H Thaler. Anomalies: The winner’s curse. *Journal of economic perspectives*, 2(1):191–202, 1988.
- [23] Richard H Thaler. *The winner’s curse: Paradoxes and anomalies of economic life*. Simon and Schuster, 2012.
- [24] Charles AE Goodhart and CAE Goodhart. *Problems of monetary management: the UK experience*. Springer, 1984.
- [25] Lilian Weng. Reward hacking in reinforcement learning. <https://lilianweng.github.io/posts/2024-11-28-reward-hacking/>, November 2024. Accessed: 2025-05-03.
- [26] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [27] Joar Skalse, Nikolaus Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.
- [28] Lukas Fluri, Leon Lang, Alessandro Abate, Patrick Forré, David Krueger, and Joar Skalse. The perils of optimizing learned reward functions: Low training error does not guarantee low regret. *arXiv preprint arXiv:2406.15753*, 2024.
- [29] Thomas Kwa, Drake Thomas, and Adrià Garriga-Alonso. Catastrophic Goodhart: regularizing RLHF with KL divergence does not mitigate heavy-tailed reward misspecification. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [30] El-Mahdi El-Mhamdi and Lê-Nguyên Hoang. On goodhart’s law, with an application to value alignment. *arXiv preprint arXiv:2410.09638*, 2024.
- [31] Alexander Bondarenko, Denis Volk, Dmitrii Volkov, and Jeffrey Ladish. Demonstrating specification gaming in reasoning models. *arXiv preprint arXiv:2502.13295*, 2025.

- [32] OpenAI. Detecting misbehavior in frontier reasoning models, March 2025.
- [33] Anthropic. Claude 3.7 sonnet system card, July 2025. Technical Report.
- [34] Imre Csiszár, Paul C Shields, et al. Information theory and statistics: A tutorial. *Foundations and Trends® in Communications and Information Theory*, 1(4):417–528, 2004.
- [35] Ananth Balashankar, Ziteng Sun, Jonathan Berant, Jacob Eisenstein, Michael Collins, Adrian Hutter, Jong Lee, Chirag Nagpal, Flavien Prost, Aradhana Sinha, Ananda Theertha Suresh, and Ahmad Beirami. InfAlign: Inference-aware language model alignment, February 2025. arXiv:2412.19792 [cs].
- [36] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.
- [37] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, et al. Rewardbench: Evaluating reward models for language modeling. *arXiv preprint arXiv:2403.13787*, 2024.
- [38] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. *Numerical mathematics*, volume 37. Springer Science & Business Media, 2006.
- [39] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- [40] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward Model Ensembles Help Mitigate Overoptimization, March 2024. arXiv:2310.02743 [cs].
- [41] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy S Liang, and Tatsunori B Hashimoto. AlpacaFarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems*, 36:30039–30069, 2023.
- [42] Zaiyan Xu, Sushil Vemuri, Kishan Panaganti, Dileep Kalathil, Rahul Jain, and Deepak Ramachandran. Distributionally Robust Direct Preference Optimization, February 2025. arXiv:2502.01930 [cs].
- [43] Zhiyuan Zeng, Jiatong Yu, Tianyu Gao, Yu Meng, Tanya Goyal, and Danqi Chen. Evaluating large language models at evaluating instruction following. *arXiv preprint arXiv:2310.07641*, 2023.
- [44] Yuchun Miao, Sen Zhang, Liang Ding, Rong Bao, Lefei Zhang, and Dacheng Tao. Inform: Mitigating reward hacking in rlhf via information-theoretic reward modeling. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [45] Rui Yang, Ruomeng Ding, Yong Lin, Huan Zhang, and Tong Zhang. Regularizing Hidden States Enables Learning Generalizable Reward Model for LLMs, October 2024. arXiv:2406.10216 [cs].
- [46] Jacek Karwowski, Oliver Hayman, Xingjian Bai, Klaus Kiendlhofer, Charlie Griffin, and Joar Max Viktor Skalse. Goodhart’s law in reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [47] Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton. Goal misgeneralization: Why correct specifications aren’t enough for correct goals. *arXiv preprint arXiv:2210.01790*, 2022.
- [48] Specification gaming: the flip side of ai ingenuity. <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>. Accessed: 2025-05-07.
- [49] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models, 2024.
- [50] Lichang Chen, Chen Zhu, Davit Soselia, Jiuhai Chen, Tianyi Zhou, Tom Goldstein, Heng Huang, Mohammad Shoeybi, and Bryan Catanzaro. Odin: Disentangled reward mitigates hacking in rlhf. *arXiv preprint arXiv:2402.07319*, 2024.
- [51] Jane Pan, He He, Samuel R Bowman, and Shi Feng. Spontaneous reward hacking in iterative self-refinement. *arXiv preprint arXiv:2407.04549*, 2024.
- [52] Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alexander Nicholas D’Amour, Krishnamurthy Dvijotham, Adam Fisch, Katherine A Heller, Stephen Robert Pfohl, Deepak Ramachandran, et al. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. In *First Conference on Language Modeling*, 2024.

- [53] Ahmed M. Ahmed, Rafael Rafailov, Stepan Sharkov, Xuechen Li, and Sanmi Koyejo. Scalable Ensembling For Mitigating Reward Overoptimisation, June 2024. *arXiv:2406.01013 [cs]*.
- [54] Alexandre Rame, Nino Vieillard, Leonard Hussenot, Robert Dadashi-Tazehoz, Geoffrey Cideron, Olivier Bachem, and Johan Ferret. WARM: On the Benefits of Weight Averaged Reward Models. In *International Conference on Machine Learning*, pages 42048–42073. PMLR, 2024.
- [55] Yuki Ichihara, Yuu Jinnai, Tetsuro Morimura, Kenshi Abe, Kaito Ariu, Mitsuki Sakamoto, and Eiji Uchibe. Evaluation of best-of-n sampling strategies for language model alignment. *Transactions on Machine Learning Research*, 2025.
- [56] Isha Puri, Shivchander Sudalairaj, Guangxuan Xu, Kai Xu, and Akash Srivastava. A probabilistic inference approach to inference-time scaling of llms using particle-based monte carlo methods. *arXiv preprint arXiv:2502.01618*, 2025.
- [57] Paria Rashidinejad and Yuandong Tian. Sail into the Headwind: Alignment via Robust Rewards and Dynamic Labels against Reward Hacking. *arXiv preprint arXiv:2412.09544*, 2024.
- [58] Zhihan Liu, Miao Lu, Shenao Zhang, Boyi Liu, Hongyi Guo, Yingxiang Yang, Jose Blanchet, and Zhaoran Wang. Provably mitigating overoptimization in rlhf: Your sft loss is implicitly an adversarial regularizer. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [59] Yuchun Miao, Sen Zhang, Liang Ding, Yuqi Zhang, Lefei Zhang, and Dacheng Tao. The energy loss phenomenon in rlhf: A new perspective on mitigating reward hacking. *arXiv preprint arXiv:2501.19358*, 2025.
- [60] Audrey Huang, Wenhao Zhan, Tengyang Xie, Jason D Lee, Wen Sun, Akshay Krishnamurthy, and Dylan J Foster. Correcting the myths of kl-regularization: Direct alignment without overoptimization via chi-squared preference optimization. In *The Thirteenth International Conference on Learning Representations*.
- [61] Xiaoying Zhang, Jean-Francois Ton, Wei Shen, Hongning Wang, and Yang Liu. Overcoming reward overoptimization via adversarial policy optimization with lightweight uncertainty estimation. *arXiv preprint arXiv:2403.05171*, 2024.
- [62] Banghua Zhu, Michael I Jordan, and Jiantao Jiao. Iterative data smoothing: Mitigating reward overfitting and overoptimization in rlhf. *arXiv preprint arXiv:2401.16335*, 2024.
- [63] Yizhou Chen, Yawen Liu, Xuesi Wang, Qingtao Yu, Guangda Huzhang, Anxiang Zeng, Han Yu, and Zhiming Zhou. Establishing reliability metrics for reward models in large language models. *arXiv preprint arXiv:2504.14838*, 2025.
- [64] Lingfeng Shen, Sihao Chen, Linfeng Song, Lifeng Jin, Baolin Peng, Haitao Mi, Daniel Khashabi, and Dong Yu. The Trickle-down Impact of Reward (In-)consistency on RLHF, September 2023. *arXiv:2309.16155 [cs]*.
- [65] Jiayi Fu, Xuandong Zhao, Chengyuan Yao, Heng Wang, Qi Han, and Yanghua Xiao. Reward Shaping to Mitigate Reward Hacking in RLHF, 2025.
- [66] Tianqi Liu, Wei Xiong, Jie Ren, Lichang Chen, Junru Wu, Rishabh Joshi, Yang Gao, Jiaming Shen, Zhen Qin, Tianhe Yu, et al. RRM: Robust reward model training mitigates reward hacking. *arXiv preprint arXiv:2409.13156*, 2024.
- [67] Chaoqi Wang, Zhuokai Zhao, Yibo Jiang, Zhaorun Chen, Chen Zhu, Yuxin Chen, Jiayi Liu, Lizhu Zhang, Xiangjun Fan, Hao Ma, et al. Beyond reward hacking: Causal rewards for large language model alignment. *arXiv preprint arXiv:2501.09620*, 2025.
- [68] Jacob Hilton, Peter Clark, et al. Measuring goodhart’s law: Towards an evaluation framework for open-ended generative models. OpenAI Blog, 2022. Accessed: 2025-01-30.
- [69] Jiahao Qiu, Yifu Lu, Yifan Zeng, Jiacheng Guo, Jiayi Geng, Huazheng Wang, Kaixuan Huang, Yue Wu, and Mengdi Wang. Treebon: Enhancing inference-time alignment with speculative tree-search and best-of-n sampling. *arXiv preprint arXiv:2410.16033*, 2024.
- [70] Ruiqi Zhang, Momin Haider, Ming Yin, Jiahao Qiu, Mengdi Wang, Peter Bartlett, and Andrea Zanette. Accelerating best-of-n via speculative rejection. In *2nd Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@ ICML 2024)*, 2024.
- [71] Pier Giuseppe Sessa, Robert Dadashi, Léonard Hussenot, Johan Ferret, Nino Vieillard, Alexandre Ramé, Bobak Shariari, Sarah Perrin, Abe Friesen, Geoffrey Cideron, et al. Bond: Aligning llms with best-of-n distillation. *arXiv preprint arXiv:2407.14622*, 2024.
- [72] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

- [73] Afra Amini, Tim Vieira, and Ryan Cotterell. Variational best-of-n alignment. *arXiv preprint arXiv:2407.06057*, 2024.
- [74] Tong Yang, Jincheng Mei, Hanjun Dai, Zixin Wen, Shicong Cen, Dale Schuurmans, Yuejie Chi, and Bo Dai. Faster WIND: Accelerating Iterative Best-of- N Distillation for LLM Alignment. In *The 28th International Conference on Artificial Intelligence and Statistics*.
- [75] Yinlam Chow, Guy Tennenholtz, Izzeddin Gur, Vincent Zhuang, Bo Dai, Sridhar Thiagarajan, Craig Boutilier, Rishabh Agarwal, Aviral Kumar, and Aleksandra Faust. Inference-Aware Fine-Tuning for Best-of- N Sampling in Large Language Models, 2024.
- [76] Alizée Pace, Jonathan Mallinson, Eric Malmi, Sebastian Krause, and Aliaksei Severyn. West-of-n: Synthetic preference generation for improved reward modeling. *arXiv preprint arXiv:2401.12086*, 2024.

This appendix is organized as follows:

- **Section A** expands on the inevitability of reward hacking, stating Theorem 1, highlighting the key assumptions, and summarizing its main consequences.
- **Section B** contains the proof of Theorem 2, including derivations of the KL–reward tradeoff and discussion of the Best-of-Poisson example.
- **Section C** discusses additional details on Theorem 3, characterizing the hacking threshold and outlining computational approaches.
- **Section D** presents additional experimental details and results.

A Inference-Time Reward Hacking

One can observe that initially intensifying optimization to a proxy objective may improve intended performance. However, beyond a certain point – which we call the *hacking threshold* – further proxy-optimization actually degrades the true reward. Theorem 1 in Section 2 formalizes this phenomenon: under very general conditions on a one-parameter family of proxy distributions π_θ , the map

$$\theta \mapsto \mathbb{E}_{X \sim \pi_\theta}[r_t(X)] := f(\theta) \quad (7)$$

can have at most one interior extremum. Thus, there is either a monotonic benefit (or disadvantage) to strengthening the proxy or exactly one “sweet spot” before reward hacking sets in.

Let $\psi(x, \theta)$ denote the score function of distribution $\pi_\theta(x)$ with density $p_\theta(x)$. Standard calculations under mild regularity conditions gives us the derivative of the true reward under π_θ :

$$f'(\theta) = \int r_t(x) \nabla_\theta p_\theta(x) dx = \int r_t(x) p_\theta(x) \nabla_\theta \log p_\theta(x) dx = \mathbb{E}_{X \sim \pi_\theta}[r_t(X) \psi(X, \theta)] \quad (8)$$

The rest of this appendix proceeds as follows. We first restate Theorem 1 in more detail (see Theorem 4). We prove the single-crossing property for the derivative of the true reward expectation by invoking variation-diminishing kernels. We then derive Corollary 2 and 3 and Lemma 1. We specialize the discussion to two concrete examples with Best-of- n and Best-of-Poisson.

Theorem 4 (Inevitability of Reward Hacking). Let $\{\pi_\theta\}_{\theta \in \Theta \subset \mathbb{R}}$ be a family of probability measures on a measurable space $(\mathcal{X}, \mathcal{A})$ with density $p_\theta(x)$ relative to a common dominating measure. Assume that

1. p_θ is *strictly TP₂* (totally positive with order 2): for every $\theta_1 < \theta_2$ and $x_1 < x_2$,

$$p_{\theta_1}(x_1) p_{\theta_2}(x_2) > p_{\theta_1}(x_2) p_{\theta_2}(x_1).$$

2. The score function $\psi(x, \theta) := \partial_\theta \log p_\theta(x)$ exists, is continuous in x , and is *strictly increasing in x* for each fixed θ .
3. Denote $f(\theta) := \mathbb{E}_{\pi_\theta}[r_t(X)]$ where $r_t : \mathcal{X} \rightarrow [0, \infty)$ is non-negative, bounded, and not identically zero.

Then the derivative $f'(\theta) = \mathbb{E}_{\pi_\theta}[r_t(X) \psi(X, \theta)]$ changes sign at most once on Θ . Consequently f is monotone or possesses exactly one interior extremum.

The main idea to establish this result is that when we use parameter θ to control inference-time methods, we create a family of densities $\{p_\theta(x)\}$ that act as positive kernels. Crucially, these kernels satisfy the strict total positivity conditions required for variation-diminishing theorems to apply. The key insight is that the expression $f'(\theta) = \mathbb{E}_\theta[r_t(X) \psi(X, \theta)]$ captures how the **rate of change** of expected reward depends on the interaction between:

- $r_t(X)$: The true reward function (which may have complex variation patterns). Our only assumption on the true reward is that it is bounded. We then translate the reward function so that it is non-negative. The boundedness assumption is a natural one in the alignment setting because real-world rewards originate from human judgments given on finite scales (e.g. star ratings, Likert scores, or normalized preference probabilities). Moreover, clipping or normalizing the reward prevents unbounded returns, improving the stability of policy updates and inference-time mechanisms.
- $\psi(X, \theta) = \partial_\theta \log p_\theta(X)$: The score function (which encodes how the distribution changes with the parameter)

The variation-diminishing theorem tells us that this interaction can have at most one sign change, which directly implies that $f'(\theta)$ can cross zero at most once, i.e., $f(\theta)$ is either monotonic or has exactly one extremum and this, in turn implies that *reward hacking follows predictable patterns*. This theoretical finding then explains the characteristic behavior of reward hacking as shown in empirical findings.

We now present the proof for Theorem 4.

Proof. Fix θ and set $h_\theta(x) := r_t(x)\psi(x, \theta)$. Because $\psi(\cdot, \theta)$ is strictly increasing, it has at most one zero; since $r_t \geq 0$, h_θ has the same single ($- \rightarrow +$) sign change in x . Strict TP₂ of p_θ and Karlin’s variation–diminishing theorem imply that $\theta \mapsto F(\theta) := \int h_\theta(x)p_\theta(x) dx = f'(\theta)$ inherits at most the same number of sign changes, namely one. $\square$

Having established the inevitability result, we next explore how it specializes in classical families via a simple corollary.

Corollary 2 (Strict MLR densities). Let $p_\theta(x)$ be *strictly monotone–likelihood–ratio* in x (i.e. $p_{\theta_2}(x)/p_{\theta_1}(x)$ strictly increases in x whenever $\theta_2 > \theta_1$). If $\psi(x, \theta) = \partial_\theta \log p_\theta(x)$ is strictly increasing in x , then all conclusions of Theorem 4 apply.

Example 1 (One-parameter exponential families). Any regular canonical exponential family

$$p_\theta(x) = \exp\{\eta(\theta)T(x) - A(\theta) + B(x)\},$$

with strictly monotone statistic T and strictly monotone natural parameter η is strict MLR and satisfies $\psi(x, \theta) = \eta'(\theta)T(x) - A'(\theta)$, which is strictly increasing in x . Hence the single–crossing property holds for every bounded non-negative $r_t \geq 0$.

The conditions are satisfied by two inference-time methods we study:

Best-of-n: The distribution corresponds to the maximum of n i.i.d. samples from the reference distribution. When proxy rewards are uniformly distributed, this yields $p_n(u) = nu^{n-1}$ for $u \in [0, 1]$. The likelihood ratio $\frac{p_{n_2}(u)}{p_{n_1}(u)} = \frac{n_2}{n_1} u^{n_2-n_1}$ is strictly increasing in u when $n_2 > n_1$, establishing strict MLR (which implies strict TP₂). The score function $\psi(u, n) = \frac{1}{n} + \log u$ is strictly increasing in u .

Best-of-Poisson: The distribution $p_\mu(u) = (\mu u + 1)e^{\mu(u-1)}$ for $u \in [0, 1]$ can be verified to satisfy strict MLR by direct computation of likelihood ratios. The score function $\psi(u, \mu) = u - 1 + \frac{u}{\mu u + 1}$ is strictly increasing in u .

Under the conditions presented in Theorem 4, we know that at most one interior extrema exists. The following corollary gives precise conditions on when such an interior extrema exists.

Lemma 1. Under the assumptions of Theorem 1 and with $r_t \in C^1$ on a neighborhood of its boundaries (0 and 1) and let $\Theta = [\theta_l, \theta_r]$,

$$\lim_{\theta \downarrow \theta_\ell} f'(\theta) = r'_t(0+) \mathbb{E}_{\theta_\ell}[X \psi(X, \theta_\ell)], \quad \lim_{\theta \uparrow \theta_r} f'(\theta) = -r'_t(1-) \mathbb{E}_{\theta_r}[(1-X)\psi(X, \theta_r)].$$

Then, a stationary point exists iff

$$\lim_{\theta \downarrow \theta_\ell} f'(\theta) \text{ and } \lim_{\theta \uparrow \theta_r} f'(\theta)$$

are of opposite sign (or one limit is 0 while the other is non-zero). By continuity, the Intermediate-Value Theorem then forces one root of f' and single–crossing rules out a second. For example, for BoN, $\psi(x, n) = 1/n + \log x$ with $\mathbb{E}_n[X \psi] = 1/(n+1)^2$, $\mathbb{E}_n[(1-X)\psi] = -1/(n+1)^2$. Hence a stationary point exists iff $r'_t(0+)$ and $r'_t(1-)$ have opposite signs, and its location $n_\star$ solves $\mathbb{E}_n[r_t(X)\psi(X, n)] = 0$.

Proof. We prove the left boundary results (the right one is identical with $x \mapsto 1-x$.) Write the first-order expansion $r_t(x) = r_t(0+) + r'_t(0+)x + R(x)$ with $R(x) = o(x)$ as $x \rightarrow 0$. Because $\mathbb{E}_\theta[\psi] = 0$,

$$f'(\theta) = r'_t(0+)\mathbb{E}_\theta[X\psi] + \mathbb{E}_\theta[R(X)\psi].$$

Strict increase of ψ implies $|X\psi| \leq C(1+X)$ on $[0, 1] \times [\theta_\ell, \theta_\ell + \rho]$, so $\mathbb{E}_\theta[X\psi] \rightarrow \mathbb{E}_{\theta_\ell}[X\psi]$ by dominated convergence. Next, fix $\delta \in (0, 1)$ and split the expectation:

$$\mathbb{E}_\theta[r_t(X)\psi] = \mathbb{E}_\theta[r_t(X)\psi \mathbf{1}_{\{X \leq \delta\}}] + \mathbb{E}_\theta[r_t(X)\psi \mathbf{1}_{\{X > \delta\}}].$$

Near 0, we have $|R(x)| \leq c_\delta x$, hence the term is bounded by $c_\delta \mathbb{E}_\theta[X|\psi|]$; choose δ so small that $c_\delta \mathbb{E}_{\theta_\ell}[X|\psi|] < \varepsilon$ and continuity keeps it $< 2\varepsilon$ for θ close enough to θ_ℓ . Away from 0, we use boundedness of r_t and local boundedness of ψ to obtain a factor $P_\theta\{X > \delta\} \rightarrow 0$.

Combining the two parts gives $\mathbb{E}_\theta[R(X)\psi] \rightarrow 0$, yielding the claimed limit. $\square$

Corollary 3 (Reward behavior for MLR samplers). Assume Lemma 1 holds and that the family $\{p_\theta\}_{\theta \in \Theta}$ is *monotone-likelihood-ratio* in $x \in (0, 1)$. Then for every $\theta \in \Theta$

$$L_\theta := \mathbb{E}_\theta[X \psi(X, \theta)] > 0, \quad R_\theta := \mathbb{E}_\theta[(1 - X)\psi(X, \theta)] < 0,$$

so

$$\text{sign } f'(\theta_\ell^+) = \text{sign } r'_t(0+), \quad \text{sign } f'(\theta_r^-) = \text{sign } r'_t(1-)$$

Single-crossing of f' implies that $f(\theta)$ can assume *exactly* one of the four shapes:

shape of $f(\theta)$	$r'_t(0+)$	$r'_t(1-)$
monotone increase	≥ 0	≥ 0
single peak then decline	> 0	< 0
monotone decrease	≤ 0	≤ 0
single valley (decline then rise)	< 0	> 0

Proof. Let p_θ be differentiable in θ with score $\psi(x, \theta) = \partial_\theta \log p_\theta(x)$. For any integrable g we have shown that:

$$\frac{d}{d\theta} \mathbb{E}_\theta[g(X)] = \mathbb{E}_\theta[g(X)\psi(X, \theta)]$$

We first use that the MLR property implies first-order stochastic dominance. For every *increasing* function g , we have that $\theta \mapsto \mathbb{E}_\theta[g(X)]$ is non-decreasing and its derivative is ≥ 0 . Choosing $g(x) = x$ gives

$$L_\theta = \mathbb{E}_\theta[X \psi(X, \theta)] = \frac{d}{d\theta} \mathbb{E}_\theta[X] > 0$$

On the other hand, $g(x) = 1 - x$ (strictly decreasing) yields:

$$R_\theta = \mathbb{E}_\theta[(1 - X)\psi(X, \theta)] = \frac{d}{d\theta} \mathbb{E}_\theta[1 - X] < 0$$

Thus $L_\theta > 0$ and $R_\theta < 0$ for every $\theta \in \Theta$. □

We have shown the conditions under which the expected value of true reward has a critical point with respect to θ . We now show the conditions under which our results extend identically if we are studying the expected value of true reward as a function of the KL divergence with respect to the reference distribution π_{θ_0} .

Lemma 2 (Score-covariance formula for $D'(\theta \parallel \theta_0)$). Let $\{p_\theta\}_{\theta \in \Theta}$ be a regular parametric family with density $p_\theta(x)$ such that

- p_θ and $\partial_\theta p_\theta$ are jointly measurable and $\partial_\theta p_\theta(x)$ is locally integrable in θ ;
- the *score* $\psi(x, \theta) := \partial_\theta \log p_\theta(x)$ is square-integrable: $\mathbb{E}_\theta[\psi^2] < \infty$.

For a fixed reference point $\theta_0 \in \Theta$ define the Kullback–Leibler divergence

$$D(\theta \parallel \theta_0) := \int p_\theta(x) \log \frac{p_\theta(x)}{p_{\theta_0}(x)} d\mu(x).$$

Then $D(\theta \parallel \theta_0)$ is differentiable and

$$\frac{d}{d\theta} D(\theta \parallel \theta_0) = \mathbb{E}_\theta[(\log p_\theta(x) - \log p_{\theta_0}(x)) \psi(X, \theta)].$$

In particular, for canonical exponential families with strictly increasing natural parameter, this simplifies to

$$\frac{d}{d\theta} D(\theta \parallel \theta_0) = (\eta(\theta) - \eta(\theta_0)) A''(\theta),$$

which is strictly positive when $\theta > \theta_0$ (and negative for $\theta < \theta_0$).

Proof. Write $g_\theta(x) := \log p_\theta(x) - \log p_{\theta_0}(x)$. Then $D(\theta||\theta_0) = \mathbb{E}_\theta[g_\theta(X)]$. For a *parameter-dependent* integrand the classical Fisher–Leibniz rule gives

$$\frac{d}{d\theta} \mathbb{E}_\theta[g_\theta(X)] = \mathbb{E}_\theta[\partial_\theta g_\theta(X)] + \mathbb{E}_\theta[g_\theta(X) \psi(X, \theta)], \quad (\dagger)$$

whenever $\partial_\theta g_\theta$ exists and an L^1 dominated–convergence bound holds (true here by the square–integrable score assumption). Since $\partial_\theta g_\theta(x) = \psi(x, \theta)$ and $\mathbb{E}_\theta[\psi] = 0$, the first term vanishes, leaving exactly

$$\frac{d}{d\theta} D(\theta||\theta_0) = \mathbb{E}_\theta[g_\theta(X) \psi(X, \theta)]$$

□

B Best-of-Poisson

In this section, we prove Theorem 2 and establish, as a consequence, that Best-of-Poisson is numerically near-optimal as compared to tilted distribution π_λ^* in terms of KL divergence. As a consequence, hedging in the optimal tilted distribution is almost equivalent to hedging with BoP, making BoP an effective inference-time stand-in for the optimal tilted distribution which can be difficult to numerically estimate. We start by establishing the BoP distribution in the uniform case.

Theorem 5 (BoP Distribution). Let $\mu > 0$ be the parameter of the Best-of-Poisson sampling method and let X_μ be the random variable representing the response selected by BoP. The probability density function $q_\mu(x)$ of X_μ is given by:

$$q_\mu(x) = (1 + \mu x) e^{\mu(x-1)}, \quad (9)$$

for $x \in [0, 1]$, where $n = n' + 1$ with $n' \sim \text{Poisson}(\mu)$.

Proof. Write $X_\mu = \max\{U_0, U_1, \dots, U_{n'}\}$ where $n' \sim \text{Poisson}(\mu)$ and $U_i \stackrel{\text{iid}}{\sim} \text{Unif}[0, 1]$. Consider $U_0 \sim \text{Unif}[0, 1]$ to be the mandatory draw to achieve a sample size of at least one.

For $x \in [0, 1]$,

$$F_\mu(x) := \Pr(X_\mu \leq x) = \Pr(U_0 \leq x) \Pr(U_i \leq x \text{ for } 1 \leq i \leq n') = x \mathbb{E}[x^{n'}] = x e^{-\mu(1-x)},$$

because $\mathbb{E}[x^{n'}] = \exp\{-\mu(1-x)\}$ is the moment-generating function of a Poisson variable evaluated at $\log x$.

Now, differentiating F_μ on $(0, 1)$ gives

$$q_\mu(x) = e^{-\mu(1-x)} + \mu x e^{-\mu(1-x)} = (1 + \mu x) e^{\mu(x-1)},$$

which extends continuously to the endpoints. A direct computation verifies $\int_0^1 q_\mu(x) dx = 1$, so q_μ is a valid density. □

Now, we can prove Theorem 2.

Theorem 6 (KL Divergence of BoP). Let X_μ be the random variable representing the response selected by BoP with parameter μ . Then:

$$D_{\text{KL}}(\pi_{\text{Poisson}}||\pi_{\text{ref}}) = \frac{e^{-\mu-1}(\text{Ei}(\mu+1) - \text{Ei}(1))}{\mu} + \log(\mu+1) - 1, \quad (10)$$

$$\mathbb{E}[X_\mu] = 1 - \frac{1}{\mu} + \frac{1 - e^{-\mu}}{\mu^2}, \quad (11)$$

where $\text{Ei}(z) = -\int_{-z}^{\infty} \frac{e^{-t}}{t} dt$ is the exponential integral function.

Proof. Mean.

$$\mathbb{E}[X_\mu] = \int_0^1 x(1 + \mu x) e^{\mu(x-1)} dx = \int_0^1 (x + \mu x^2) e^{\mu(x-1)} dx$$

With the substitution $u = \mu(x-1)$, we get that

$$\int_0^1 x e^{\mu(x-1)} dx = \frac{1}{\mu^2} \int_{-\mu}^0 (u + \mu) e^u du = \frac{\mu - 1 + e^{-\mu}}{\mu^2}$$

$$\int_0^1 x^2 e^{\mu(x-1)} dx = \frac{1}{\mu^3} \int_{-\mu}^0 (u + \mu)^2 e^u du = \frac{\mu^2 - 2\mu + 2 - 2e^{-\mu}}{\mu^3}$$

Hence

$$\mathbb{E}[X_\mu] = \frac{\mu - 1 + e^{-\mu}}{\mu^2} + \mu \frac{\mu^2 - 2\mu + 2 - 2e^{-\mu}}{\mu^3} = 1 - \frac{1}{\mu} + \frac{1 - e^{-\mu}}{\mu^2}$$

KL divergence. Because $\log q_\mu(x) = \log(1 + \mu x) + \mu(x - 1)$,

$$D_{\text{KL}}(q_\mu \| U) = \int_0^1 q_\mu(x) \log(1 + \mu x) dx + \mu[\mathbb{E}[X_\mu] - 1]$$

To compute the integral, set $t = 1 + \mu x$:

$$\int_0^1 q_\mu(x) \log(1 + \mu x) dx = \frac{e^{-\mu-1}}{\mu} \int_1^{\mu+1} t e^t \log t dt$$

Integration by parts ($f = \log t$, $dg = t e^t dt$) yields

$$\int t e^t \log t dt = \frac{1}{2} t^2 e^t \left(\log t - \frac{1}{2} \right) - \frac{1}{2} \text{Ei}(t) + C$$

hence

$$\int_0^1 q_\mu(x) \log(1 + \mu x) dx = \log(\mu + 1) - \frac{1 - e^{-\mu}}{\mu} + \frac{e^{-\mu-1}}{\mu} [\text{Ei}(\mu + 1) - \text{Ei}(1)]$$

The resulting term becomes:

$$D_{\text{KL}}(q_\mu \| \text{Unif}) = \frac{e^{-\mu-1}}{\mu} [\text{Ei}(\mu + 1) - \text{Ei}(1)] + \log(\mu + 1) - 1 \quad \square$$

Now that we have derived the exact formulas for BoP's KL divergence and expected reward, we can establish that BoP provides a practical approximation to the optimal tilted distribution with negligible performance loss.

Theorem 7 (Near-Optimality of BoP). Let π_{Poisson} be the distribution induced by Best-of-Poisson with parameter $\mu > 0$, and let π_λ^* be the optimal KL-constrained tilted distribution with parameter $\lambda > 0$, defined as:

$$\pi_\lambda^*(x) = \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)}, \quad (12)$$

where $Z(\lambda)$ is the normalization constant. For any given expected reward level, there exists a μ for BoP and a λ for the tilted distribution such that $\mathbb{E}_{X \sim \pi_{\text{Poisson}}}[r_p(X)] = \mathbb{E}_{X \sim \pi_\lambda^*}[r_p(X)]$, and the KL divergence gap between these distributions is given by:

$$D_{\text{KL}}(\pi_{\text{Poisson}} \| \pi_{\text{ref}}) - D_{\text{KL}}(\pi_\lambda^* \| \pi_{\text{ref}}) = \frac{e^{-(\mu+1)}(\text{Ei}(\mu+1) - \text{Ei}(1))}{\mu} + \ln\left(\frac{(\mu+1)(e^\lambda - 1)}{\lambda}\right) - \frac{\lambda e^\lambda}{e^\lambda - 1} \quad (13)$$

In particular, numerical evaluation shows that for all μ , if $\lambda > 0$ is chosen so that $\mathbb{E}_{X \sim \pi_{\text{Poisson}}}[r_p(X)] = \mathbb{E}_{X \sim \pi_\lambda^*}[r_p(X)]$, then the KL-gap satisfies

$$0 \leq D_{\text{KL}}(\pi_{\text{Poisson}} \| \pi_{\text{ref}}) - D_{\text{KL}}(\pi_\lambda^* \| \pi_{\text{ref}}) \leq 7 \times 10^{-3}.$$

That is, Best-of-Poisson achieves nearly the optimal trade-off between expected reward and KL divergence from the reference distribution.

Proof. Theorem 2 above established that

$$D_{\text{KL}}(\pi_{\text{Poisson}} \| \pi_{\text{ref}}) = \frac{e^{-\mu-1}(\text{Ei}(\mu+1) - \text{Ei}(1))}{\mu} + \log(\mu+1) - 1 \quad (14)$$

Now, we need to derive the KL divergence between the optimal tilted distribution π_λ^* and the reference distribution π_{ref} . Recall that the optimal tilted distribution with parameter λ is given by:

$$\pi_\lambda^*(x) = \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)} \quad (15)$$

where $Z(\lambda) = \int_{\mathcal{X}} \pi_{\text{ref}}(x) e^{\lambda r_p(x)} dx$ is the normalization constant.

The KL divergence is defined as:

$$D_{D_{\text{KL}}}(\pi_{\lambda}^* \| \pi_{\text{ref}}) = \int \pi_{\lambda}^*(x) \log \frac{\pi_{\lambda}^*(x)}{\pi_{\text{ref}}(x)} dx \quad (16)$$

Substituting $\pi_{\lambda}^*(x) = \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)}$ in the KL divergence gives:

$$D_{D_{\text{KL}}}(\pi_{\lambda}^* \| \pi_{\text{ref}}) = \int \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)} \log \frac{e^{\lambda r_p(x)}}{Z(\lambda)} dx \quad (17)$$

$$= \int \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)} [\lambda r_p(x) - \log Z(\lambda)] dx \quad (18)$$

$$= \lambda \int \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)} r_p(x) dx - \log Z(\lambda) \int \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)} dx \quad (19)$$

The second integral equals 1 since π_{λ}^* is a probability distribution. For the first integral, we recognize it as the expected value of $r_p(X)$ under the distribution π_{λ}^* :

$$\mathbb{E}_{\pi_{\lambda}^*}[r_p(X)] = \int r_p(x) \pi_{\lambda}^*(x) dx = \int r_p(x) \frac{\pi_{\text{ref}}(x) e^{\lambda r_p(x)}}{Z(\lambda)} dx \quad (20)$$

Therefore:

$$D_{D_{\text{KL}}}(\pi_{\lambda}^* \| \pi_{\text{ref}}) = \lambda \mathbb{E}_{\pi_{\lambda}^*}[r_p(X)] - \log Z(\lambda) \quad (21)$$

This shows that the KL divergence is given by the expected reward minus the log partition function. We can further simplify by noting that the derivative of $\log Z(\lambda)$ with respect to λ gives us the expected reward:

$$\frac{d}{d\lambda} \log Z(\lambda) = \frac{1}{Z(\lambda)} \frac{d}{d\lambda} \int \pi_{\text{ref}}(x) e^{\lambda r_p(x)} dx = \frac{1}{Z(\lambda)} \int \pi_{\text{ref}}(x) e^{\lambda r_p(x)} r_p(x) dx = \mathbb{E}_{\pi_{\lambda}^*}[r_p(X)] \quad (22)$$

Therefore:

$$D_{D_{\text{KL}}}(\pi_{\lambda}^* \| \pi_{\text{ref}}) = \lambda \frac{d}{d\lambda} \log Z(\lambda) - \log Z(\lambda) \quad (23)$$

Which is a standard result for exponential families namely, that the KL divergence equals the Bregman divergence of the log partition function. For our specific case where π_{ref} is the uniform distribution on $[0, 1]$ and $r_p(x) = x$ (after the probability integral transform), we have:

$$Z(\lambda) = \int_0^1 e^{\lambda x} dx = \frac{e^{\lambda} - 1}{\lambda} \log Z(\lambda) = \log \left(\frac{e^{\lambda} - 1}{\lambda} \right) \quad (24)$$

The derivative is:

$$\frac{d}{d\lambda} \log Z(\lambda) = \frac{e^{\lambda}}{e^{\lambda} - 1} - \frac{1}{\lambda} \quad (25)$$

So:

$$D_{D_{\text{KL}}}(\pi_{\lambda}^* \| \pi_{\text{ref}}) = \lambda \left(\frac{e^{\lambda}}{e^{\lambda} - 1} - \frac{1}{\lambda} \right) - \log \left(\frac{e^{\lambda} - 1}{\lambda} \right) \quad (26)$$

$$= \lambda \frac{e^{\lambda}}{e^{\lambda} - 1} - 1 - \log \left(\frac{e^{\lambda} - 1}{\lambda} \right) \quad (27)$$

After further algebraic manipulation:

$$D_{D_{\text{KL}}}(\pi_{\lambda}^* \| \pi_{\text{ref}}) = \lambda - 1 + \frac{\lambda}{e^{\lambda} - 1} - \log \left(\frac{e^{\lambda} - 1}{\lambda} \right) \quad (28)$$

This shows that the difference in KL is given by

$$D_{\text{KL}}(\pi_{\text{Poisson}} \parallel \pi_{\text{ref}}) - D_{\text{KL}}(\pi_{\lambda}^* \parallel \pi_{\text{ref}}) \quad (29)$$

$$= \frac{e^{-\mu-1}(\text{Ei}(\mu+1) - \text{Ei}(1))}{\mu} + \log(\mu+1) - 1 - \left(\lambda - 1 + \log\left(\frac{\lambda}{e^{\lambda}-1}\right) + \frac{\lambda}{e^{\lambda}-1} \right) \quad (30)$$

To numerically verify the near-optimality claim, we solve the equation $\mathbb{E}_{X \sim \pi_{\text{BoP}}}[r_p(X)] = \mathbb{E}_{X \sim \pi_{\lambda}^*}[r_p(X)]$ using Newton's method to find λ as a function of μ . For each value of μ , we then evaluate the KL divergence gap:

$$D_{\text{KL}}(\pi_{\text{BoP}} \parallel \pi_{\text{ref}}) - D_{\text{KL}}(\pi_{\lambda}^* \parallel \pi_{\text{ref}}) = \frac{e^{-(\mu+1)}(\text{Ei}(\mu+1) - \text{Ei}(1))}{\mu} + \ln\left(\frac{(\mu+1)(e^{\lambda}-1)}{\lambda}\right) - \frac{\lambda e^{\lambda}}{e^{\lambda}-1} \quad (31)$$

□

Numerical evaluation confirms that this difference is bounded by approximately 7×10^{-3} across the entire range of μ as in Figure 2. This validates that Best-of-Poisson indeed provides a practically equivalent approximation to the optimal tilted distribution with negligible computational overhead.

C Proof of Theorem 3

In Section 4, we present an efficient way to find the optimal hacking threshold. We restate Theorem 3 and prove it.

Theorem 8 (Hacking Threshold Characterization). Let r_t be a true reward oracle and $\theta^\dagger$ be the hacking threshold from Definition 1. For each inference-time method, $\theta^\dagger$ is characterized by the following conditions:

$$\text{For BoN, } n^\dagger \text{ satisfies: } \nabla_{\alpha} \mathbb{E}_{u \sim \text{Beta}(\alpha, 1)}[r_t(u)] = \int_0^1 r_t(u) (1 + n^\dagger \log u) u^{n^\dagger-1} du = 0, \quad (32)$$

$$\text{For SBoN, } \lambda^\dagger \text{ satisfies: } \nabla_{\lambda} \mathbb{E}_{u \sim f_{\lambda}}[r_t(u)] = \mathbb{E}_{U_1, \dots, U_n}[\text{Cov}(r_t(V), V | U_1, \dots, U_n)]|_{\lambda=\lambda^\dagger} = 0, \quad (33)$$

$$\text{For BoP, } \mu^\dagger \text{ satisfies: } \nabla_{\mu} \mathbb{E}_{u \sim f_{\mu}}[r_t(u)] = \mathbb{E}_{u \sim f_{\mu^\dagger}} \left[r_t(u) \left(u + \frac{u-1}{\mu^\dagger u + 1} \right) \right] = 0. \quad (34)$$

Proof. We now consider each mechanism separately:

Hedging in Best-of- n . In BoN, we approximate the integer n via a continuous parameter α by placing a $\text{Beta}(\alpha, 1)$ prior on u . Its density is $f_{\alpha}(u) = \alpha u^{\alpha-1}$, so $\psi(u, \alpha) = \partial_{\alpha}[\ln \alpha + (\alpha-1) \ln u] = \frac{1}{\alpha} + \ln u$. Thus, the optimality condition becomes

$$\mathbb{E}_{u \sim \text{Beta}(\alpha, 1)} \left[r_t(u) \left(\frac{1}{\alpha} + \ln u \right) \right] = 0 \iff \int_0^1 r_t(u) (1 + n \log u) u^{n-1} du = 0, \quad (35)$$

which one solves for α to pick an effective sample size. In practice, this equation must be solved numerically. We do this by discretizing $[0, 1]$ into M evenly-spaced points $u_i = \frac{i-0.5}{M}, i = 1, \dots, M$ and forming the Riemann-sum residual

$$R(\alpha) = \sum_{i=1}^M g(u_i) \left(\frac{1}{\alpha} + \ln u_i \right) u_i^{\alpha-1} \Delta u,$$

where $\Delta u = 1/M$. Since $\Delta u > 0$, the root $R(\alpha) = 0$ is equivalent to the hedging condition. Then, one applies any root-finding method, see, e.g., [38], to locate the unique solution α^* . Finally, the discrete sample size is chosen as $N = \lceil \alpha \rceil$, the integer nearest α^* , which gives the effective BoN that exactly hedges against proxy-score hacking.

Hedging in Soft Best-of- n . Unlike BoN and BoP, SBoN does not admit a simple closed-form density due to its sampling mechanism. In SBoN, we first sample n responses $X_1, \dots, X_n$ from the reference distribution, then select response X_i with probability $\frac{e^{\lambda r(X_i)}}{\sum_{j=1}^n e^{\lambda r(X_j)}}$. The resulting distribution is:

$$\pi_{n, \lambda}(x) = \mathbb{E}_{X_1, \dots, X_{n-1} \sim \pi_{\text{ref}}} \left[\pi_{\text{ref}}(x) \cdot \frac{e^{\lambda r(x)}}{\frac{1}{n} \left(e^{\lambda r(x)} + \sum_{i=1}^{n-1} e^{\lambda r(X_i)} \right)} \right]$$

We can now derive the hedging condition $\frac{\partial}{\partial \lambda} \mathbb{E}_{u \sim f_\lambda} [r_t(u)] = 0$. For the case of proxy reward percentiles, remember that the SBoN sampling mechanism works as follows

1. Sample $U_1, \dots, U_n \sim \text{Uniform}[0, 1]$ independently
2. Select index Z with probability $P(Z = i | U_1, \dots, U_n) = \frac{e^{\lambda U_i}}{\sum_{j=1}^n e^{\lambda U_j}}$
3. Return $V = U_Z$

Then, to start, we compute $\frac{\partial p_i}{\partial \lambda}$:

$$\frac{\partial p_i}{\partial \lambda} = \frac{\partial}{\partial \lambda} \left[\frac{e^{\lambda U_i}}{S} \right] = \frac{U_i e^{\lambda U_i} \cdot S - e^{\lambda U_i} \cdot \frac{\partial S}{\partial \lambda}}{S^2} \quad (36)$$

Since $\frac{\partial S}{\partial \lambda} = \sum_{j=1}^n U_j e^{\lambda U_j}$:

$$\frac{\partial p_i}{\partial \lambda} = \frac{e^{\lambda U_i}}{S} \left[U_i - \frac{\sum_{j=1}^n U_j e^{\lambda U_j}}{S} \right] = p_i \left[U_i - \sum_{j=1}^n U_j p_j \right] \quad (37)$$

Now, we compute the derivative of the expected reward:

$$\frac{\partial}{\partial \lambda} \mathbb{E}_{u \sim f_\lambda} [r_t(u)] = \mathbb{E}_{U_1, \dots, U_n} \left[\frac{\partial}{\partial \lambda} \sum_{i=1}^n r_t(U_i) p_i \right] \quad (38)$$

$$= \mathbb{E}_{U_1, \dots, U_n} \left[\sum_{i=1}^n r_t(U_i) \frac{\partial p_i}{\partial \lambda} \right] \quad (39)$$

$$= \mathbb{E}_{U_1, \dots, U_n} \left[\sum_{i=1}^n r_t(U_i) p_i \left(U_i - \sum_{j=1}^n U_j p_j \right) \right] \quad (40)$$

Expanding this expression:

$$= \mathbb{E}_{U_1, \dots, U_n} \left[\sum_{i=1}^n r_t(U_i) p_i U_i - \sum_{i=1}^n r_t(U_i) p_i \sum_{j=1}^n U_j p_j \right] \quad (41)$$

$$= \mathbb{E}_{U_1, \dots, U_n} \left[\sum_{i=1}^n r_t(U_i) p_i U_i - \left(\sum_{i=1}^n r_t(U_i) p_i \right) \left(\sum_{j=1}^n U_j p_j \right) \right] \quad (42)$$

The expression inside the expectation is exactly the conditional covariance

$$\text{Cov}(r_t(V), V | U_1, \dots, U_n) = \mathbb{E}[r_t(V) \cdot V | U_1, \dots, U_n] - \mathbb{E}[r_t(V) | U_1, \dots, U_n] \cdot \mathbb{E}[V | U_1, \dots, U_n], \quad (43)$$

where:

$$\mathbb{E}[r_t(V) | U_1, \dots, U_n] = \sum_{i=1}^n r_t(U_i) p_i \quad (44)$$

$$\mathbb{E}[V | U_1, \dots, U_n] = \sum_{i=1}^n U_i p_i \quad (45)$$

$$\mathbb{E}[r_t(V) \cdot V | U_1, \dots, U_n] = \sum_{i=1}^n r_t(U_i) U_i p_i \quad (46)$$

Therefore:

$$\frac{\partial}{\partial \lambda} \mathbb{E}_{u \sim f_\lambda} [r_t(u)] = \mathbb{E}_{U_1, \dots, U_n} [\text{Cov}(r_t(V), V | U_1, \dots, U_n)]. \quad (47)$$

Unlike BoN and BoP, SBoN does not admit a simple closed-form density due to its sampling mechanism. The hedging condition $\nabla_\lambda \mathbb{E}_{u \sim f_\lambda} [r_t(u)] = 0$ corresponds to $\mathbb{E}_{U_1, \dots, U_n} [\text{Cov}(r_t(V), V | U_1, \dots, U_n)] = 0$, where the expectation is taken over the random samples $U_1, \dots, U_n$ and the conditional covariance is computed with respect to the softmax selection probabilities. This condition must be evaluated numerically using the following procedure:

1. For a given λ , generate M independent realizations of $(U_1^{(m)}, \dots, U_n^{(m)}) \sim \text{Uniform}[0, 1]^n$ for $m = 1, \dots, M$.
2. For each realization m , compute the conditional covariance:

$$C^{(m)} = \text{Cov}(r_t(V), V | U_1^{(m)}, \dots, U_n^{(m)}) = \sum_{i=1}^n r_t(U_i^{(m)}) U_i^{(m)} p_i^{(m)} - \left(\sum_{i=1}^n r_t(U_i^{(m)}) p_i^{(m)} \right) \left(\sum_{j=1}^n U_j^{(m)} p_j^{(m)} \right) \quad (48)$$

$$\text{where } p_i^{(m)} = \frac{e^{\lambda U_i^{(m)}}}{\sum_{j=1}^n e^{\lambda U_j^{(m)}}}.$$

3. Estimate the derivative as $\hat{D}(\lambda) = \frac{1}{M} \sum_{m=1}^M C^{(m)}$.
4. Use root-finding methods (e.g., bisection or Newton's method) to locate λ^* where $\hat{D}(\lambda^*) = 0$.

Hedging in Best-of-Poisson. Here, one draws a $\text{Poisson}(\mu)$ number of samples (plus one) and selects the proxy-maximal u . For uniform u , the density is $f_\mu(u) = (\mu u + 1)e^{-\mu(1-u)} = u - 1 + \frac{u}{\mu u + 1}$, giving $\psi(u, \mu) = \partial_\mu \ln f_\lambda(u) = \partial_\lambda [\ln f_n(u) + \lambda u - \ln Z(\lambda)] = u - \mathbb{E}_{u \sim f_\lambda(v)}$. The hedging equation

$$\nabla_\mu \mathbb{E}_{\pi_\mu} [r_y(X)] = \mathbb{E}_{u \sim f_\mu} \left[r_t(u) \left(u - 1 + \frac{u}{\mu u + 1} \right) \right] = 0 \quad (49)$$

In an analogous way to the previous hedging equations, we solve the residual

$$R(\mu) = \sum_{i=1}^M g(u_i) s_\mu(u_i) \frac{\tilde{w}_i(\mu)}{Z'(\mu)} \Delta u \quad (50)$$

to locate μ^* such that $R(\mu^*) = 0$. This μ^* is the Poisson optimal hedge. $\square$

D Experimental Details

In this section, we provide additional details on our experimental setup. We provide our code here.

D.1 Toy Example

In Figure 1, we present a toy example with one-dimensional rewards defined on $[0, 1]$. We set the proxy reward to be $r_p(x) = x$ and the gold reward

$$r_t(x) = \frac{x^p(1-x)}{C}, \quad C = \left(\frac{p}{p+1} \right)^p \frac{1}{p+1}$$

where C is a normalization constant so that the gold reward is bounded between 0 and 1 for convenience. We choose this gold reward as the reward under the Best-of- n distribution has a simple closed-form solution. We set $p = 12$ and we find the expected value of true reward and the KL divergence with respect to the reference distribution under four mechanisms:

1. **Tilted Distribution:** Exponential tilting of the gold reward, $Q_\lambda(x) \propto \exp(\lambda r_t(x))$ and of the proxy reward, $Q_\lambda(x) \propto \exp(\lambda r_p(x))$.
2. **Best-of- n (BoN):** Selection of the maximum of n i.i.d. uniform draws.
3. **Soft Best-of- n (SBoN):** Softmax-based sampling of n i.i.d. uniform draws with inverse temperature λ .
4. **Best-of-Poisson (BoP):** Selection of the maximum of n i.i.d. uniform draws where n is drawn from a Poisson distribution with rate μ .

Below we detail each one in turn.

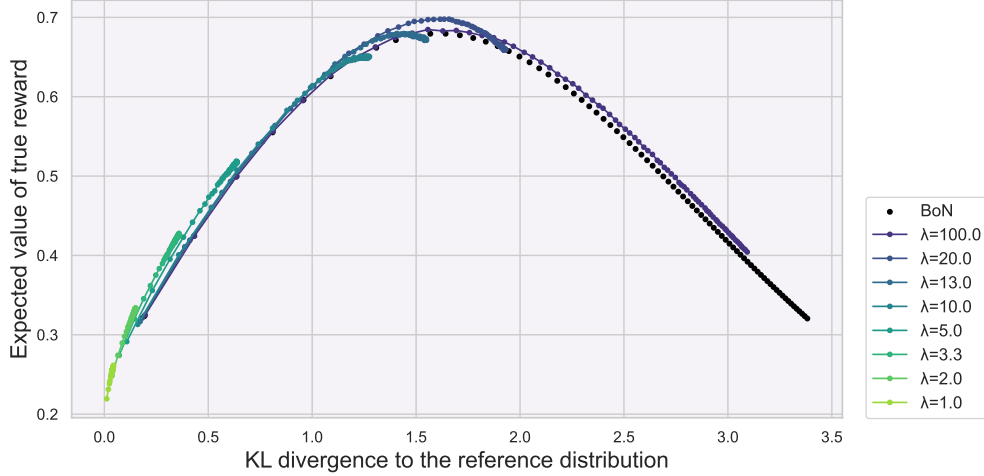


Figure 6: Expected value of true reward versus KL divergence to the reference distribution using both Best-of- n and Soft Best-of- n in the setup presented in Sec. D.1. For the same range of $n = \{1, \dots, 80\}$, Soft BoN *softens* the winner’s curse with an appropriate λ while attaining a competitive reward vs KL tradeoff.

Tilted Distribution We first consider tilting with respect to the gold reward, defined as:

$$Q_\lambda(x) = \frac{e^{\lambda r_t(x)}}{Z(\lambda)}, \quad Z(\lambda) = \int_0^1 e^{\lambda r_t(x)} dx,$$

We then compute

$$\mathbb{E}_{Q_\lambda}[r_t] = \frac{1}{Z(\lambda)} \int_0^1 r_t(x) e^{\lambda r_t(x)} dx, \quad D_{\text{KL}}(Q_\lambda \| U[0, 1]) = \lambda \mathbb{E}_{Q_\lambda}[r_t] - \ln Z(\lambda).$$

All integrals are evaluated via numerical quadrature on $[0, 1]$. Analogously, letting

$$Z_{\text{proxy}}(\lambda) = \int_0^1 e^{\lambda r_p(x)} dx = \begin{cases} 1, & \lambda = 0, \\ \frac{e^\lambda - 1}{\lambda}, & \lambda \neq 0, \end{cases}$$

we define

$$Q_\lambda^{\text{proxy}}(x) = \frac{e^{\lambda x}}{Z_{\text{proxy}}(\lambda)},$$

and compute $\mathbb{E}_{Q_\lambda^{\text{proxy}}}[r_t]$ and its KL with respect to the reference distribution (the uniform distribution).

Best-of- n For n i.i.d. draws $X_{1:n} \sim U[0, 1]$, the max has density $n x^{n-1}$. One shows

$$\mathbb{E}[r_t(\max X_{1:n})] = \frac{n}{C(n+p)(n+p+1)}, \quad D_{\text{KL}} = \ln n - 1 + \frac{1}{n}.$$

Soft-BoN We draw $m = 1.2 \times 10^5$ samples of $(X_{1:n})$, apply a softmax with temperature λ , and numerically estimate $\mathbb{E}[r_t]$ and D_{KL} . We sweep λ over 600 log-spaced values in $[10^{-4}, 10^4]$.

Best-of-Poisson (BoP) We treat the number of draws as distributed from a Poisson distribution $n \sim \text{Poisson}$ with sampling density $q_\lambda(x) = (\lambda x + 1) e^{\lambda(x-1)}$, from which $\mathbb{E}[r_t]$ and D_{KL} are again computed by numerical quadrature. We sweep μ over 800 uniformly-spaced values in $[0, 300]$.

We apply HedgeTune as presented in Alg. 4 for BoN and BoP. In both cases, we attain an operating point which corresponds exactly to the true hacking threshold as shown in Fig. 1. The success of this algorithm hinges on Theorem 1 which guarantees an existence of (at least) one hacking threshold. However, this guarantee does not hold for Soft BoN. Therefore, the optimization problem becomes much more challenging, and using vanilla estimators for the density causes numerical instabilities when applied for Soft BoN.

Below is a reddit POST and the corresponding SUBREDDIT and TITLE.
 Write a both precise and concise summary of the contents of the post.

SUBREDDIT: r/relationships
TITLE: I [23F] have just come out of 8 year relationship. Feel like I don't know how to date/flirt. Scared will grow old with many cats. Any advice?
POST: This is my first post so please be kind :)

I know that lots of people often feel confused when they come out of a long-term relationship. They think they have forgotten how to be single, or how to flirt/date.

I am one of these people.

The problem is, my relationship started when I had just turned 16. I have never been single—as an adult. That might sound silly. But the only time I have ever flirted or dated was as an over-confident, hormone-riddled teenager.

Now I have a pretty demanding job, responsibilities blah blah... And I just don't know how to this!

I'm in no way in a rush to get into a new relationship, but that doesn't mean I want to be completely alone in the meantime.

If anyone has experienced anything similar, or just generally has some advice, it would be greatly appreciated!

Summary:

Figure 7: Prompt template used to summarize a selected Reddit post

Summary 1: "I am a 23/F looking for advice on how to find potential lovers. I think I find no one, and it frustrates me. Any advice?"

Summary 2: "I just came out of a relationship, a really long one. I know I'm young but I feel like I must be naive and lack experience. Please give me anything you can!"

Summary 3: "Teenage dating syndrome. Don't know-how-to- avoid it. Care to chat and/or share stories about your dating/relationship experiences?"

Figure 8: Example of generated summaries of the Reddit post shown in Fig. 7

D.2 Synthetic setup

We randomly select a single Reddit post from the validation split of the `trl-internal-testing/tldr-preference-trl-style` dataset on Hugging Face. Using this prompt (see Figure 7), we generate 60,000 candidate summaries from a supervised fine-tuned Pythia-1.4B model (`cleanrl/EleutherAI_pythia-1b-deduped__sft__tldr` on HuggingFace) via ancestral sampling with temperature $T = 1$ and a top- k filter of $k = 50$ (see examples of candidate summaries in Figure 8). After generating candidate summaries, we annotate them by scoring with a reward model trained on summarization task. Concretely, we use `cleanrl/EleutherAI_pythia-1b-deduped__reward__tldr` found on Hugging Face.

Using this annotated data, we would like to empirically observe the *winner's curse*. Methods like Best-of- n that aggressively optimize for the proxy are particularly vulnerable to this phenomenon. We artificially miscalibrate the proxy reward model at the extreme high-end of its scores. Concretely, let P_α denote the top $\alpha\%$ -percentile of proxy scores. For any response whose proxy score lies in P_α , we replace its proxy score by a *negatively* linear function of its true reward (so that higher true-reward responses receive lower proxy scores). All other responses retain their true

reward as the proxy. Formally,

$$r_p(x) = \begin{cases} a - b r_t(x), & \text{if } r_t(x) \geq P_\alpha, \\ r_t(x), & \text{otherwise,} \end{cases}$$

where $a, b > 0$ are chosen so that this mapping spans the same range as the original scores. We then map both distributions to uniform percentiles via their sorted ranks, yielding two arrays in $[0, 1]$ amenable to direct comparison. We consider α in the set $\{80, 85, 90, 95\}$, simulating increasingly egregious misalignment at the top end of the proxy. This synthetic setup let us measure how each selection strategy (BoN, SBoN, and BoP) degrades in true-reward performance as the proxy becomes more misleading among its highest-ranked candidates.

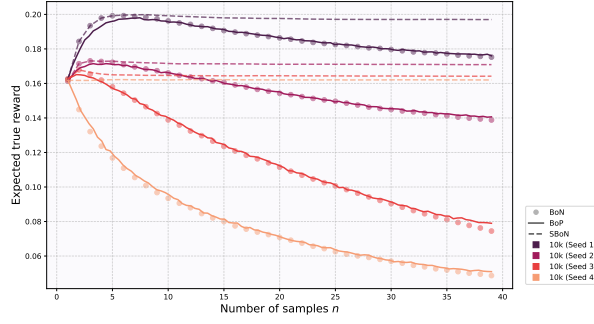
We define our sampling grids as $n \in \{1, \dots, 1000\}$ (BoN), $\lambda \in [0, 20]$ (SBoN), $\mu \in [0, 50]$ (BoP) each discretized into 300, 150, and 250 points respectively. For every parameter setting and for each bootstrap replicate ($B = 300$), we apply the selection mechanism, record the chosen item’s true reward, and check whether this true reward exceeds the true reward of a randomly selected response by at least 10%. We aggregate these outcomes to estimate, for each combination, the probability of at least 10% true-reward improvement, and compute 95% confidence intervals via a normal approximation. We consider this probability as a calibrated reward. In Figure 3, we present the proxy and true (calibrated) rewards using BoN and BoP where we vary the average number of samples n . Moreover, we find the value of n corresponding to the peak true reward for each case using HedgeTune. Meanwhile, in Figure 4, we compare the performance of Soft Best-of- n mechanism with an optimized temperature $\lambda^\dagger$ to Best-of- n . We do so by plotting the gap between the (calibrated) reward attained by SBoN and that attained with BoN. With larger n , Best-of- n is more likely to fall victim to the winner’s curse, making hedging an effective solution to improve performance.

D.3 Reward hacking in the wild

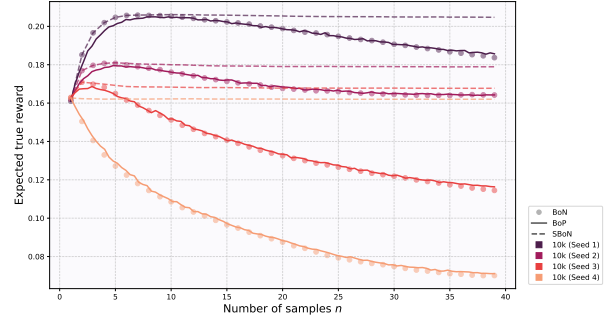
To observe reward hacking *in the wild*, we follow the setup of Coste et al. [40]. We first use an annotated dataset provided by [40] which contains 1,000 prompts from the validation split of AlpacaFarm dataset, along with 12,600 response generations per prompt from a 1.4b fine-tuned Pythia model. Each prompt-response pair is labeled with the AlpacaFarm reward-model-human to give ‘gold’ scores. Next, we would like to train proxy reward models on the preferences of this true reward. We randomly sample a prompt with two responses from the annotated dataset and curate a dataset of the form (prompt, chosen, rejected) where the chosen response is the response with the higher gold reward score. We follow this procedure to curate four datasets with varying sizes (10k, 20k, 46k, 80k). For each dataset, we consider two variants: one with no label noise and one with random 25% label noise. Next, we use the code kindly provided by the authors of [40] in their Github repository to train proxy reward models with the different datasets over four random seeds (1, 2, 3 and 4) using their default hyperparameters (e.g., 10^{-5} learning rate and five epochs). Lastly, we score the annotated dataset using the trained proxy reward models. The end result is a set of 800 prompts, 12 600 responses per prompt, along with gold and proxy scores for each prompt-response pair.

While reward hacking can appear without label noise (see left panels of Figures 9 and 10), reward hacking is more pronounced with label noise as expected. Moreover, reward hacking is more apparent when the proxy reward is trained on **less** data. One potential explanation is that, with fewer training examples, the proxy is less well-calibrated and its estimation errors vary more sharply across inputs. In that case, a small n might produce a deceiving reward gain. In contrast, errors may surface early on with a large training dataset, so true reward declines immediately as sampling increases. In cases of reward hacking, we see that SBoN with an appropriately chosen λ can (1) achieve the maximum reward achieved by BoN/BoP and (2) mitigate reward hacking, as shown with the reward almost flatlining after it reaches its peak value. We also witness cases where the proxy reward **always** misaligns with the gold reward, causing a collapse of true reward from the onset of BoN. In that case, the optimal hedging behavior is a uniform selection over responses, which is recovered with $n = 1$ for BoN or $\lambda = 0$ for SBoN.

Interestingly, we observe instances of what we call **reward grokking** as shown in the right panels of Figures 10 and 12, where the true reward decreases or flat-lines across low- to mid-range sample counts, only to undergo a sudden uptick at higher sample regimes, revealing a delayed but apparent realignment of proxy and true objectives. We leave detailed investigation of reward grokking and its implications for hedging strategies to future work.

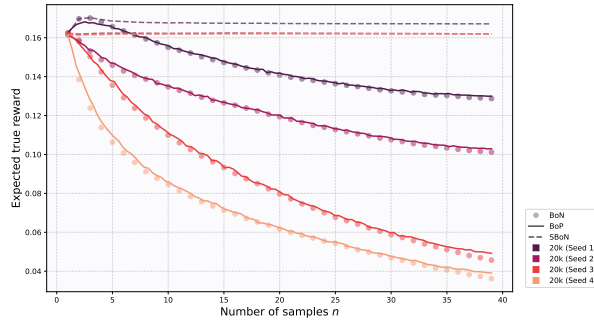


(a) No label noise

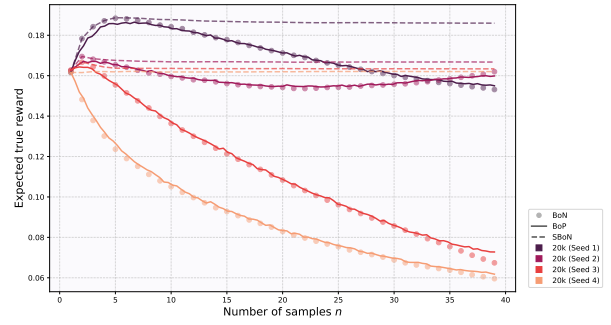


(b) 25% label noise

Figure 9: Expected true-reward vs. average number of samples with proxy trained on 10 000 examples: (a) without label noise; (b) with 25% label noise.

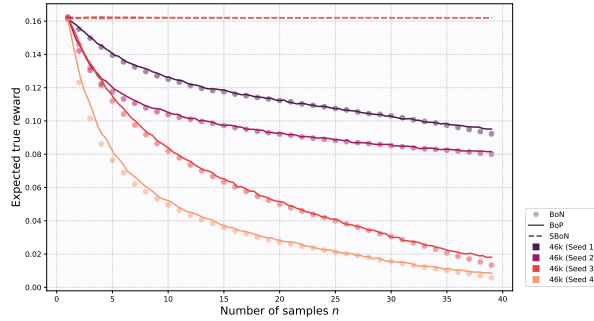


(a) No label noise

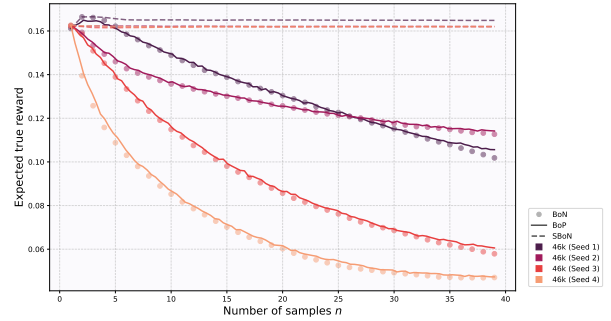


(b) 25% label noise

Figure 10: Expected true-reward vs. average number of samples with proxy trained on 20 000 examples: (a) without label noise; (b) with 25% label noise.



(a) No label noise



(b) 25% label noise

Figure 11: Expected true-reward vs. average number of samples with proxy trained on 46 000 examples: (a) without label noise; (b) with 25% label noise.

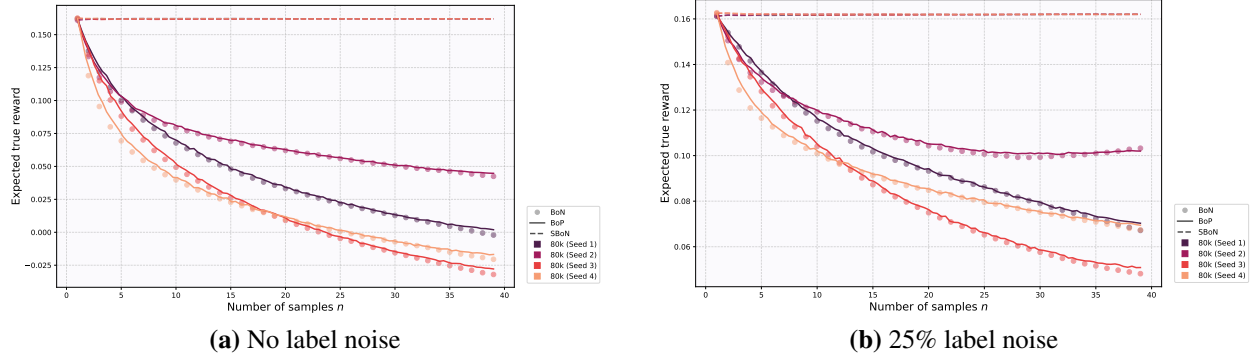


Figure 12: Expected true-reward vs. average number of samples with proxy trained on 80 000 examples: (a) without label noise; (b) with 25% label noise.